

PROJECT PROPOSAL & TECHNICAL LEDGER

OpenDerja_TN

An open, regionally balanced written corpus of Tunisian Derja — where Tunisians write, correct and confirm their own language, and what comes out is citable, downloadable, and still alive in five years.

Firas Lamouchi · solo project, open-source and open to public contribution once the MVP ships

Planning started **14 April 2026** · build started **28 May 2026** · updated **25 August 2026**

CONTENTS

What's in here

The argument, then the machine, then what I haven't solved.

01	The proposal	11	Machine assistance — deferred
02	Infrastructure and cost	12	Ingestion and scraping
03	Security and abuse	13	Standardisation, transliteration, matching
04	Legal, licensing, consent	14	The translator
05	What I haven't solved	15	Frontend and design system
06	Four regions, six axes	16	Every page, specified
07	Data model	17	The admin dashboard
08	The task engine	18	The public API
09	Accounts and permissions	19	Export and publication
10	Backend architecture	20	Build sequence

How to read this. §1 is the pitch. §2–5 are the practical reality check — cost, security, legal footing, and the open problems — so you know what you're actually signing up for before the schema starts. §6–19 are the build reference: the data model, the rules, the decisions I've already made so I don't have to make them again at 2am. §20 is the order I'm building in. The companion *Architecture & Engineering Blueprint* covers how the code is allowed to be structured, and *VOICE.md* covers how the site talks to people, which is a different job from how this document talks to me.

This document sits in between two others I haven't written yet: a non-technical version for people who just want to understand what this is and why, and a developer-facing one for anyone who wants to join and build. This one is for both audiences at once, which is why it reads the way it does.

Nothing here is final. This is the plan as I understand it after thinking about it for about two months. I already know some of it will turn out wrong once I'm actually building, and more of it will change once the repo is public and other people start arguing with parts of it. That's expected, not a failure of the plan — it gets updated at the next big milestone, not pretended into permanence now.

01

The proposal

What this is, what it isn't, and why I'm doing it anyway.

A platform that keeps running, openly licensed, where Tunisians write, correct and confirm their own regional Derja. Out of it comes a written corpus and a lexicon — balanced by region, layered by granularity, tied to real attestations, and available to download, browse, or query.

1.1 What it is

- **Text first, voice by design.** The platform runs on text from day one and carries voice attestation from a future update — not bolted on later. The consent model, the schema, and the tables for it (§7.11) exist from the first migration, before a single recording is collected.
- **Regionally balanced on purpose.** Coverage per region is a number I track, publish and recruit against — not something I hope works out.
- **Alive, not archived.** Academic corpora stop the day the paper ships. This one keeps growing.
- **CODA-compliant.** Uses CODA-TUN and NOTA conventions instead of inventing another spelling standard.
- **Usable, not just downloadable.** The corpus is exposed as a tool people have a reason to open, not only a file researchers have a reason to cite.

1.2 What it isn't

Saying this out loud isn't modesty. It's what stops a researcher who knows the field from closing the tab in the first thirty seconds.

- Not the first standardisation of Tunisian Arabic. CODA-TUN (LREC 2014) got there; NOTA replaced it.
- Not the first Tunisian corpus. Tunisiya, TARIC, TSAC, TEC, TunSwitch, TEDxTN all exist.
- Not the first Arabic dialect crowdsourcing platform. MyVoice/QCRI, DARIJA-C, KALAM'DZ came first.
- Not competing with institutions that have funding and staff.

SO WHAT'S LEFT

No existing platform combines all of this in one place: **text now and voice by design later; community confirmation instead of one authority's word; region and era tagged together, so old Derja sits next to today's four regions instead of being a separate archive; a working translator, not just a downloadable file; open data anyone can pull; and a shared place to publish and discuss the research itself.** Every one of those pieces exists somewhere on its own. Nowhere do they exist together, continuously maintained, as one open service. That's the whole argument.

1.3 What counts as success

LEVEL	WHAT IT LOOKS LIKE
Minimum	A published, citable, openly licensed, regionally tagged written Derja dataset that didn't exist before, with honest coverage stats — and a public tool built on it that a non-technical Tunisian can use without reading anything first.
Good	People contributing from all four regions without me pushing. People arriving through the translator and staying. Other researchers using the data. A working relationship with an established Arabic-NLP group.
Strong	Cited in Tunisian NLP work. A regional classifier and translation-memory service people actually use. Enough backing that maintenance isn't just me, forever.

1.4 Why I'm building it

This is a non-profit effort, by design, from the first line of code. No company, no equity, no plan to ever charge for access, nothing here built to be sold. A public-good language resource doesn't get a paywall.

Twelve million people write this language every day — in texts, in comments, in the middle of French sentences — and there is almost no proper record of it. My grandmother says a word for something in the northwest that I have never heard anywhere else, and when she stops saying it, it's gone. Nobody wrote it down. That happens constantly, and nobody is on it.

"Somebody should write this down" has been true for decades. I'd rather be the person who did some of it. Even the minimum version leaves behind a dataset that didn't exist and a tool people can use — that's a real thing to have made, and it stands on its own even if nothing after phase 2 ever happens.

The other reason is just as real: it's a serious piece of engineering in a domain I care about, and I want to have built it. Both reasons hold at once.

The phases are scoped so the value survives even if the later ones never happen. If I run out of steam, that produces something finished, not something abandoned halfway.

02

Infrastructure and cost

Not decided yet, on purpose — and a much easier decision without a model on the box.

WHERE HOSTING STANDS

Hosting is still undecided. This section will be updated once it's decided. What this turns out to be determines what it needs to run on, so the specs stay open until the build is further along. Deployment is a phase 2 problem (§20). Nothing in phases 1–3 is blocked on it.

2.1 Hosting

Provider undecided — to be updated here once it's picked. No permanent physical location, so self-hosted hardware isn't an option in any version of this; the decision is between a handful of small European or ARM-friendly VPS providers, sized against measured usage once the build is further along rather than guessed now.

2.2 Deployment shape

```
VPS (provider TBD – same topology regardless of which one)
├─ cloudflared    tunnel – no inbound ports open on the box at all (§3.1)
├─ caddy          TLS, reverse proxy, rate limiting, hostname routing
├─ frontend       Next.js (public site + dashboards)
├─ backend        NestJS – API boundary, guard chain (§10.1)
├─ worker         background jobs, capped concurrency (§10.3)
├─ postgres       single instance – the whole asset
├─ pgbouncer      connection pooling in front of postgres (§10.2)
├─ redis          cache + sessions only, not the job queue (§10.3)
├─ nlp            Python, health-check only, phase 1 (§11)
├─ processing     Rust, health-check only, phase 1 (§7.11)
└─ backups        nightly pg_dump, encrypted, -> off-box object storage
```

Ten containers instead of four — the backend split into its own process, pgBouncer and Redis added, and the two future-phase services showing up early as empty shells. All still multi-arch, so ARM against x86 isn't a decision I have to make in advance. Explicit memory limits stay on every one of them, which matters more now than it did at four services, not less.

The two health-check-only services and Redis are genuinely light — none of them hold real data or do real work yet, so the memory jump from adding them is small next to what a live ML model would cost (§11). pgBouncer itself is cheap; what it protects against is Postgres running out of connections, not memory. The number that actually matters is still whatever the VPS ends up being (§2.1) — this shape fits comfortably inside the small tier, it just isn't the four-box diagram it used to be.

2.3 Backups

Domain name isn't picked yet — that gets settled once the build is done and I know what's actually running.

BACKUPS, WHICH ARE NOT NEGOTIABLE

The corpus is the only irreplaceable thing here. I can rewrite the code in a weekend. I cannot rewrite months of community annotation.

- Nightly pg_dump, compressed and **encrypted before it leaves the box** — the dump carries emails, ip_hash values and session ids, exactly what §4.3 works hard to be careful about, so it can't travel in the clear.
- Weekly snapshot export to Hugging Face — a public release and an off-site backup in one
- Provider snapshots on (a couple of euro a month, obviously worth it)

- Restore tested quarterly — a scheduled `verify-backup-restore.yml` workflow (§10.0) runs it automatically, not a reminder on my calendar I can snooze. An untested backup isn't a backup.
- Whole stack in `docker-compose.yml`, so moving providers takes under an hour

2.4 Cost

Not worked out yet. There's no point pricing a box before I know what's actually going to run on it — cost gets figured out at launch, once hosting and the rest of this shape are actually decided, not guessed at now.

2.5 Object storage — also undecided

Backups and snapshot exports need somewhere that isn't the application server, and that's undecided alongside hosting. Today's requirement is small — nightly dumps, weekly exports, comfortably inside anyone's free tier. The requirement if voice happens (§7.11) is a different question entirely, and it's the one that should decide this rather than the current one.

The axis that matters for audio isn't storage price — cheap everywhere. It's **egress**, because serving clips back costs real money and providers differ by an order of magnitude on it. Deciding this on today's backup needs would mean deciding it on the wrong criterion.

CANDIDATE	POSITION
Cloudflare R2	Zero egress fees. The default candidate the moment voice is on the table.
Backblaze B2	Cheap storage, egress costs something. Fine for backups.
Provider-native storage	Cheapest if it ships from the same provider as the VPS anyway, not always S3-compatible the same way.
Plain S3	Works, most expensive on egress, no reason to pick it first.

Decided in phase 2 alongside hosting, sized for backups, revisited before any audio work starts.

2.6 Email

Reviewer invites and email verification need reliable delivery, and self-hosted mail from a fresh VPS IP lands in spam reliably — this bites hard on deploy day if it isn't planned for. Resend is primary, with SMTP as fallback rather than a second provider to configure (§10.0, §9.11): roughly €0–10/month at this volume.

2.7 Not doing

AWS / Azure

Kubernetes

managed Postgres

CDN

multi-region

GPU instances

always-on inference

If I want cloud provider experience for certification, that's a separate free-tier sandbox with throwaway resources. It doesn't go on this project's uptime and bill.

03

Security and abuse

A threat model sized to what will actually happen, not what sounds impressive.

3.1 What I'm actually worried about

THREAT	LIKELIHOOD	WHAT STOPS IT
Spam submissions	High	Rate limits by session and ip_hash; leaderboard counts approvals so spam earns nothing; bulk-revert by session
Deliberate data poisoning	Medium	Nothing becomes canonical on one person's say-so; consensus needs several annotators; everything reversible
Offensive or personal content posted publicly	Medium	Flag button on every permalink (§16.4), feeding the corrections review lane
Scraping a source that isn't actually permitted	Medium	ToS read required before any source goes live, not just robots.txt (§12.5)
Queue starving the frontend	Medium	Capped pg-boss poll interval and concurrency; bulk work off-peak (§10.3)
Publication comments becoming unbounded moderation work	Medium	Held for approval below trust 1, and the feature is switched off if it costs more than it returns (§7.10)
Reviewer account compromise	Low	Argon2id, httpOnly sessions, 2FA on admin, full audit log (§9.7)
Unauthorized inbound access	Low	Eliminated structurally, not mitigated — no open ports on the VPS at all. Everything arrives through Cloudflare Tunnel (§10.0); there's no port to scan or attack in the first place.
Data loss	Low, fatal	§2.3
Legal challenge over scraped or stored material	Low–Medium, revised up	Per-record provenance and licence; a tighter source list after §12's correction; source quarantine and purge
Retention complaint under GDPR	Low	Tested anonymisation routine that scrubs identifiers and orphans the text (§4.3)

3.2 Baseline

Rate limiting at Caddy, per IP anonymous and per key for the API. All input validated by shared zod schemas at the boundary. Parameterised queries only — Prisma handles it, no raw string interpolation anywhere. CSP headers, no inline scripts. Secrets in the environment, never committed, rotated on any suspicion. Admin subdomain IP-restricted where I can manage it. Dependencies pinned, advisory scanning on.

04

Legal, licensing, consent

Two licences, picked separately, and one checkbox you can't add later.

4.1 Two licences

ASSET	LICENCE	REASONING
Code	MIT or Apache-2.0	Permissive. Adoption matters more than control.
Dataset	CC-BY-SA or CC-BY-NC — still deciding	Not about protecting revenue — there isn't any. It's about stopping someone else from taking what contributors built for free, walling it off, and selling it back to them. Share-alike or non-commercial both close that door; plain-permissive doesn't. Needs a real decision, not a default.

Code credit lives separately from corpus credit. `docs/CONTRIBUTORS.md` names people who work on the codebase; the public `/contributors` leaderboard (§16.1) credits people who work on the corpus. Different contribution, different audience — folding the two together would bury the much larger group doing the actual data work under a handful of commit authors.

4.2 Consent

An explicit checkbox at submission: this material can go into an openly licensed dataset, including commercial use. Two lines of UI. Without it no contributed material is ever safely usable commercially, and you can't collect consent later once contributors have moved on.

That consent covers the record as a whole, not just the raw text — including a reviewer later attributing an origin to a contributor's word, or tagging it vulgar, archaic, or regional. Those are judgments made by someone else, published alongside the contribution, and "this material can go into an openly licensed dataset" is written to mean the annotated record, not the bare string someone typed. Worth being explicit about rather than leaving as an assumption.

THE VOICE CLAUSE SHIPS WITH THE FIRST CONTRIBUTION, YEARS EARLY

The checkbox carries a separate voice clause from day one, even though recording doesn't exist and won't for a long time (§7.11). Text consent does not cover voice, and it can't be collected retroactively — the same argument twice as hard, because a recording identifies a person in a way a sentence doesn't. Default is text-only. Voice is a second, explicit opt-in.

4.3 Revocation — the one thing that does hard-delete

TWO RULES IN THIS DOCUMENT COLLIDE HERE

§17.2 says nothing hard-deletes — soft delete plus an audit entry, because the dataset is the whole asset. But hosting in the EU puts this under GDPR. If someone revokes consent, keeping their `ip_hash` and email around is a problem a soft-delete flag doesn't fix. A row marked deleted is still a row kept.

The way out is separating two things that were being treated as one: **the person** and **the language**. Personal data can genuinely be erased. The linguistic contribution isn't personal data once it's no longer attached to anyone.


```

on consent revocation (hourly job: anonymise_revoked)
  HARD scrub, irreversible, no audit copy kept:
    submission_meta.contributor_email, contributor_name, ip_hash, session_id -> null
    users.email, users.display_name -> null (if they had an account)
    audit_log.ip_hash -> null for that actor; audit_log.diff scrubbed
  KEPT, permanently orphaned:
    the corpus item's text, its tags/tokens/links, its translations, its region/era/register,
    its dataset_split
  RECORDED:
    documents.rights_status -> 'revoked'; an audit entry with NO identifier – action, timestamp, count

```

- The text stays, the person goes. Published snapshots aren't affected — they never carried identifiers (§19.2).
- Irreversible and that's the point: admin-only, typed confirmation, rate-limited.
- Full removal is honoured on request; revocation defaults to anonymisation.

Voice is the exception to the exception. When recordings exist (§7.11), revoking voice consent deletes the object and the media row outright — it can't fall back to anonymisation the way text does, because a recording is biometric-adjacent. That's the second and last place anything is genuinely destroyed, designed in now before any recording exists.

4.4 Scraped material

Stored with its source and licence flag, kept out of the commercially clean tier by default. Public isn't a licence — scraped text is research-use unless there's evidence otherwise (§12.2). Removal requests honoured through the flag mechanism (§16.4) and source quarantine. No personal data from social sources — author identifiers stripped at ingest.

4.5 Crediting prior work

CODA-TUN, NOTA and any corpus I reuse get cited on the methodology page and in the dataset card.

05

What I haven't solved

Written down, because a risk you haven't named isn't one you're managing.

#	PROBLEM	WHERE IT STANDS
1	Hosting undecided. Will be updated here once it's decided.	Left open until the build is done and I can measure. Much lower stakes without a model on the box.
2	Dataset licence. Permissive, share-alike, or non-commercial.	Has to be decided before the first public release.
3	Region boundaries are a working hypothesis, not settled dialectology.	Multi-region tagging (§6.4) lets the data show me where boundaries actually fall.
4	Reviewer recruitment. Four regions need reviewers from four regions and I have no network for it.	Unsolved, the most serious non-technical risk. Routing (§8.5) limits damage, doesn't fix it.
5	Regional ground truth. Self-reported region is the only signal, and unreliable — people move.	Helped by asking origin separately from residence. Not solved.
6	Books and print need Arabic OCR with RTL layout handling.	Deferred. The main route to era: historical.
7	Staying alive. Depends on one person's unpaid effort continuing.	Can't be fully mitigated. Scope discipline is the only real defence.
8	Sahel as one region — Sfax may be distinct enough from Sousse to split.	Watch annotator disagreement, split only if the data demands it.
9	Reviewer queue fatigue without a dialect filter — noise leakage rises from roughly 20–30% to 40–50% of what reaches review (§12.3).	Accepted trade for now. Small batches, disable low-yield sources. Whether to pull a filter forward from phase 4 is unresolved — revisit if the queue outruns reviewers.
10	Scraper IP durability — one server, one address, HTML sources will ban it eventually.	Reduced by proxy budget and backoff (§12.5). Not eliminated.
11	Translator cold start — until the lexicon has coverage, most lookups return "not attested" and it looks broken.	Seed with hand-written entries across all four regions before launch; frame every miss as a prompt, not a failure (§14.4).
12	Object storage undecided , and the deciding criterion — egress for voice — is a phase 5 concern.	Pick something for backups now, revisit before audio (§2.5).
13	Voice consent can't be retrofitted , and voice revocation is the second exception to no-hard-delete.	Consent clause ships with the first contribution (§4.2). Deletion path designed before any recording exists (§4.3).
14	Publication comments are unbounded work — the one surface that can generate effort that isn't corpus effort.	Held for approval below trust 1, switched off entirely if it costs more than it returns (§7.10).

On the commercial side. I'm not counting on it — see §1, I'm not even trying. Everything below phase 2 is scoped so the value survives even if the rest of this list stays exactly this long forever.

06

Four regions, six axes

Get this wrong and everything downstream is wrong with it.

6.1 Regions — flat, no subregions

VALUE	COVERS
northwest	Kef, Jendouba, Siliana, Kasserine, the western highlands
north	Tunis, Bizerte, Nabeul, Ariana, Manouba, Ben Arous
sahel	Sousse, Monastir, Mahdia, Sfax, the coastal centre
south	Gabès, Medenine, Tataouine, Tozeur, Kebili, Douz, Gafsa

Four values, no subregions until there's data to justify them. The subregion structure exists from day one and sits empty — adding it later is a migration on one column, not a schema change.

6.2 Six independent axes

This is the decision everything hangs off. Region, era and setting aren't the same kind of thing and must never be squashed into one list.

AXIS	VALUES	NOTE
scope	pan_tunisian · regional	Whether it's marked at all, not how widely it's used
region	a set of: northwest · north · sahel · south	One or more. Empty when pan-Tunisian. §6.4.
subregion	empty (reserved)	Deferred
era	contemporary · historical	"Old Derja" lives here, not in region
setting	urban · rural · unknown	Cuts across region, doesn't follow it
register	neutral · formal · vulgar · archaic	Without it the output isn't usable for much. Display policy in §16.4.

"Northwest + rural + historical" is a real thing you can ask for. Four flat buckets couldn't say it — and that combination is exactly the material nobody has collected, which is most of the reason this exists.

6.3 Granularity — another separate axis

	PAN-TUNISIAN	REGIONAL
word	shared vocabulary	a northwest word for an everyday object
sentence	phrasing used everywhere	a construction that marks where you're from
paragraph	neutral connected text	connected speech with regional marking

6.4 Region is a set, because dialect is a continuum

Isoglosses don't respect buckets. A rural word can be shared between the northwest and the south and be unknown in Tunis. Coastal vocabulary can cover the north and the sahel and stop there. Neither is pan-Tunisian, and neither belongs to one region.

With a single-value column every case like that has two bad options: pick one region and throw away a real attestation, or call it pan-Tunisian and throw away the fact that it's marked at all. Both wreck the coverage numbers, which are the thing this project claims to be good at. So:

- **scope is markedness.** Does this carry a regional signal at all? That really is binary.
- **region is extent.** Which regions attest it, one to four. All four isn't the same claim as pan-Tunisian — "used everywhere and marked in each" differs from "unmarked".

So there's no single-value region column anywhere. Corpus items carry several `tag` rows on the same span (§7.3); lexicon variants use a `lexicon_variant_regions` table (§7.5), which also gives each region its own attestation count.

This also lets the region-boundary question (§5, problem 3) answer itself. If northwest and south keep getting tagged together, that's evidence about where the boundary actually falls. A single-value column would have destroyed that evidence at insert time, every time.

07

Data model

One structure with span-level annotation, because the levels bleed into each other.

7.1 The joining problem

A sentence is often marked as regional by one word in it. A paragraph can quote a variety it isn't written in. Three independent tables can't hold that, so it's one structure with annotations attached to character spans.

7.2 Documents and corpus items

```
documents
  id, source_id fk -> sources
  external_ref      text          -- video id, url, isbn; null for contributions
  title, raw_body   text          -- raw_body is the original, never touched again
  region_hint       enum          -- weak signal from the source, not a fact; single-valued on purpose
  era_hint, license enum
  rights_status     enum          -- granted | revoked | not_applicable (§4.3)
  ingested_at       timestampz

corpus_items
  id, document_id fk, parent_id fk -- parent null at top level
  unit              enum          -- paragraph | sentence | phrase      (no token)
  position          int          -- order within parent
  text              text          -- immutable once inserted
  script            enum          -- arabic | arabizi | mixed | latin
  canonical_form     text          -- CODA form, null until standardised
  canonical_map      jsonb        -- offset alignment: text -> canonical_form
  match_key          text          -- normalised for search, never shown (§13.5)
  rule_version       int
  char_offset        int          -- offset in parent, for reconstruction
  dataset_split      enum          -- train | dev | test; set at insert, never changed
  version            int          -- bumped on any accepted change (§8.1)

index (document_id, unit, position)
index (parent_id)
index gin (match_key gin_trgm_ops)
```

Contributions get wrapped in a synthetic one-corpus-item document pointing at the `contribution` source. That's deliberate rather than tidy: `license`, `rights_status` and provenance live on `documents` and nowhere else, so "can this be exported commercially?" is one lookup against one table whatever the origin. A nullable `document_id` would mean copying those columns onto corpus items and splitting that check in two forever.

`region_hint` stays a single enum even though §6.4 spent real effort making `region` a set everywhere else. That's deliberate, not an oversight: a hint is a guess about where a whole source probably comes from — a channel, a forum's general userbase — before anyone has read a word of it. It's allowed to be wrong and singular because nothing downstream trusts it; the actual `region` claim, which does need to be a set, lives on the tags attached to spans (§7.3), not here.

WHY TOKENS AREN'T ROWS HERE

The obvious move is a third level under sentences. It breaks the moment you standardise anything. Converting to CODA changes word count and every offset after the first edit — Arabizi "*bch nemchi*" becomes *باش نمشي*, clitics expand, two tokens turn into three. Every token row and every annotation pointing at one quietly stops lining up. Nothing errors. The data just starts lying.

So tokens are their own table (tokens), anchored to offsets in `text`. `text` never changes, so those offsets stay right forever. A separate `links` table is the one bridge from a token to the lexicon — `token_id` is unique there, so nothing stops one token pointing at two different lexicon entries, a constraint a single shared table couldn't express at all.

Standardisation never edits `text`. It writes a second version alongside, plus `canonical_map` — the offset alignment the rule engine produces anyway (§13.4). Any span on the original projects onto the canonical form through that map, and since the map carries its `rule_version`, changing rules regenerates both together.

7.3 Tokens, links, and tags — on spans, not levels

This is what sorts out the contamination. A paragraph tagged `south` can hold a three-word span tagged as a northern borrowing. Both true, both stored, no argument.

Three tables, not one polymorphic annotations table — the single-table version is where the naming fog crept in, because a `type: region` row and a `type: token` row have nothing in common except sharing a table. Splitting them out turned out to be a real bug fix, not just cosmetic: it's what let `links.token_id` be declared unique, below.

```
tokens                                -- a span inside one corpus item's text
  id, corpus_item_id fk
  char_start, char_end  int
  surface_text  text    -- the literal token text
  annotator_id fk      -- the 'system' user for machine-run tokenisation
  is_machine  bool
  confidence  numeric(3,2)
  index (corpus_item_id)

links                                -- the ONE bridge from a token to the lexicon
  id, token_id fk UNIQUE      -- one link per token, never two
  lexicon_entry_id fk, lexicon_variant_id fk null
  annotator_id fk, is_machine bool, confidence numeric(3,2)
  index (lexicon_entry_id)
  index (lexicon_variant_id)

tags                                -- everything else: scope, region, era, setting,
                                -- register, code_switch, sense, quality
  id, corpus_item_id fk
  char_start, char_end  int
  kind  enum  -- scope | region | era | setting | register
              -- | code_switch | sense | quality
  value  text
  annotator_id fk      -- the 'system' user for machine suggestions
  is_machine  bool
  confidence  numeric(3,2)

  index (corpus_item_id, kind, char_start, char_end)
  index (annotator_id)
```

Tokenising is its own pass, inserting one `tokens` row per token — re-tokenising means inserting rows, not rebuilding a tree. A `region` tag is one row *per region* — tagging a span northwest and south is two rows, which is how §6.4's set semantics work without an array column.

THREE THINGS THIS GIVES FREE

Code-switching — a French loanword is a span with kind: `code_switch`, value: `fr`. **Corpus and lexicon join up** — a links row pointing a token at a lexicon entry turns "this sentence attests the northwest form" into one query.

Disagreement survives — two annotators write two tags rows; consensus is worked out from them, never written over them.

7.4 Consensus — derived, never destructive

Keyed by span, not by corpus item, and that matters more than it sounds. One sentence routinely carries several tags of the same kind — three `code_switch` spans, two spans with different regional identities. (Links don't need this — one token has at most one link, so there's nothing to reach agreement over.) Key it by corpus item and kind alone and computing consensus for the second span overwrites the first. The more carefully someone tags a sentence, the more of their work the table throws away.

```
tag_consensus          -- materialised, refreshed when tags are written
  corpus_item_id fk, kind enum, char_start int, char_end int
  agreed_values         text[]          -- a set: region can agree on more than one
  agreement             numeric(3,2)    -- 1.0 = unanimous
  annotator_n           int
  needs_adjudication    bool
  computed_at           timestampz

PRIMARY KEY (corpus_item_id, kind, char_start, char_end)
index (needs_adjudication) where needs_adjudication
```

- **Whole-item tags still get a span.** register, era, scope are written with `char_start = 0, char_end = length(text)`. Never null — Postgres doesn't treat nulls as equal in a unique key, so a nullable span would let duplicates through in exactly the place uniqueness is the point.
- **Agreement runs over a set.** Two people who both say "northwest and south" agree. One saying "northwest" and one saying "northwest and south" partly agree — the truthful reading, and a useful signal for the boundary question.

7.5 Lexicon — three layers

```
lexicon_entries         -- the meaning, region-agnostic
  id, gloss_en, gloss_fr, gloss_msa
  domain enum          -- everyday | food | admin | agriculture | kinship | ...
  granularity enum, pos enum, notes

lexicon_variants        -- one realisation of a lexicon entry
  id, lexicon_entry_id fk
  scope, era, setting, register      -- single-valued axes only
  canonical_form text
  match_key text                  -- §13.5
  confidence numeric(3,2)

lexicon_variant_regions -- extent: one row per attesting region (§6.4)
  lexicon_variant_id fk ON DELETE CASCADE, region enum
  attestation_count int, confidence numeric(3,2)
  PRIMARY KEY (lexicon_variant_id, region)
  index (region, attestation_count desc)

lexicon_forms           -- a written rendering of a variant
  id, lexicon_variant_id fk, text, script, match_key
  is_canonical bool, frequency int
  unique (lexicon_variant_id, script) where is_canonical
```

Why three layers. *The question the product exists to answer — "here are four ways people say this, which is this?" — becomes a single join. Flattened, you can't ask it. Three layers also make false friends expressible: one written form reaching two lexicon entries through two variants. That happens constantly, and a flat model corrupts it silently.*

7.6 Word origin

Derja's layering is its history — Amazigh substrate, Arabic superstrate, then French, Italian, Turkish and Spanish arriving through occupation, trade and migration. Recording which layer a word comes from turns a wordlist into a document of that history, and it's the most interesting thing on the site to anyone who isn't a linguist. No existing Tunisian corpus carries systematic etymology tagging.

It hangs off `lexicon_entries`, not `lexicon_variants` — origin is a property of the word's history, not of how Kef says it versus Douz. The exception is real but rare: sometimes regions borrowed *different* words for the same meaning from different sources. That's arguably two lexicon entries, and I'd rather let the rare case carry a note than duplicate the column onto variants for a minority of entries.

```
lexicon_origins          -- most lexicon entries have none, and that's fine
  id, lexicon_entry_id fk ON DELETE CASCADE
  origin                 enum
  source_form            text null    -- 'écurie', 'tanwart' – the word it came from
  source_lang            text null
  note                   text null    -- the derivation, in prose, with a citation if there is one
  proposed_by            fk, status enum -- proposed | confirmed | disputed
  confidence              numeric(3,2)
  index (lexicon_entry_id); index (origin) where status = 'confirmed'

origin_enum:
  arabic                 -- direct, recognisably MSA
  arabic_derived          -- from Arabic but shifted in form or meaning
  french                 amazigh      italian
  turkish                 spanish      other
  unknown                 -- explicitly recorded as unknown
```

The enum lives in shared with the others (§10.1). Two things about it:

- **Italian and Turkish are there from the start because you hit them fast.** Tunisian has a real Italian layer through Sicily and the ports, and an Ottoman administrative and food layer. Leave them out and contributors file them under `other`, and you never cleanly get them back.
- **unknown isn't the same as no row.** No row means nobody looked. A row saying unknown means somebody looked and couldn't say. The second is information.

Examples

LAYER	DERJA	FROM	MEANING
amazigh	فكرون fakrun	ifker	tortoise
amazigh	نواره nawwara	tanwart	flower
french	فرشيطة farshiṭa	fourchette	fork
french	بيرو birou	bureau	desk, office
french	تريسيّتي trisiti	électricité	electricity
french	كوري kouri	écurie	stable
italian	كروسة karrusa	carrozza	cart
italian	بانكة banka	banca	bank
italian	سيمانة simana	settimana	week
turkish	طبسي ṭabsi	tepsi	plate, tray
turkish	بقلاوة baklawa	baklava	—
arabic_derived	باهي bahi	بِهْ classical: splendid	shifted to ordinary "good, fine, OK"

THE BEST ENTRY ON THE PAGE ISN'T A WORD

The suffix -ji (جي) comes from the Turkish agentive *-ci*, and Tunisian applies it to Arabic roots — قهواجي *gahwaji*, coffee seller. Derja didn't just borrow Turkish words, it borrowed a piece of Turkish grammar and still uses it productively. That gets its own note on /origins.

Disputed

DERJA	THE ARGUMENT
سبيطار sbiṭar — hospital	French <i>hôpital</i> or Italian <i>ospedale</i> ? Both plausible, both argued.
فيشته fishta — celebration	French <i>fête</i> or Italian <i>festa</i> ?
برشا barsha — a lot	The most argued word in Tunisian. Turkish, Arabic and others all have advocates.

Three disputed entries next to the confirmed ones are worth more than fifty tidy ones. They tell a visiting linguist I know etymology is contested, and they're an invitation to argue — which is engagement I don't have to manufacture.

NEVER REQUIRED, NEVER BLOCKING

Etymology spawns no task, counts toward no completeness metric, and a lexicon entry without an origin row isn't incomplete. Someone knows how their grandmother says a word; they usually don't know it came from Ottoman Turkish, and asking would either produce guesses or stop them contributing. It's opt-in knowledge from whoever happens to have it — so there's no origin field on the contribution form. It lives on the lexicon entry page as "add what you know", never in the path between arriving and submitting (§9.6).

One rights catch: a derivation copied out of a published etymological dictionary is someone's scholarship, and the etymology export tier (§19.2) makes that a redistribution question. So origins are either contributor knowledge or cited, and note carries the citation when there is one.

7.7 Translations — explicit keys, not polymorphic

One phrase can have several valid translations, so translations are rows. But *how* they point at their target matters as much as that they exist.

The tempting shape is `target_type` plus a bare `target_id` uuid. Postgres can't constrain that at all: no foreign key, no delete behaviour, no guarantee the id points at anything real. Delete a variant and its translations become orphans *nothing in the database can catch*. They pile up quietly and eventually ship inside an export.

```
translations
  id uuid pk
  corpus_item_id      uuid REFERENCES corpus_items(id) ON DELETE CASCADE
  lexicon_entry_id    uuid REFERENCES lexicon_entries(id) ON DELETE CASCADE
  lexicon_variant_id  uuid REFERENCES lexicon_variants(id) ON DELETE CASCADE
  target_lang enum, text, source enum, translator_id fk
  is_preferred bool, quality_score numeric(3,2)

CONSTRAINT single_translation_target CHECK (
  (corpus_item_id IS NOT NULL)::int + (lexicon_entry_id IS NOT NULL)::int
  + (lexicon_variant_id IS NOT NULL)::int = 1 )

index (corpus_item_id, target_lang) where corpus_item_id is not null
index (lexicon_entry_id, target_lang) where lexicon_entry_id is not null
index (lexicon_variant_id, target_lang) where lexicon_variant_id is not null

-- one preferred per language, per target kind
unique (corpus_item_id, target_lang) where is_preferred and corpus_item_id is not null
unique (lexicon_entry_id, target_lang) where is_preferred and lexicon_entry_id is not null
unique (lexicon_variant_id, target_lang) where is_preferred and lexicon_variant_id is not null
```

A bit more verbose than the polymorphic version, and worth it — integrity becomes the database's job instead of something the app is trusted to remember at 1am. All three preferred-indexes are needed, not one: the lexicon is exactly where a duplicate "preferred French gloss" would hurt, because the translator reads it directly.

The one place I keep polymorphism, on purpose. *corrections* (§7.8) still uses `target_table` + `target_id`, because a correction can target nearly any field of nearly any table and three nullable columns would become fifteen. Accepted trade: it can't cascade and gets validated in app code. Survivable because corrections are a workflow record, not corpus content, and because nothing hard-deletes (§4.3).

7.8 Supporting tables

sources	id, kind (youtube forum book subtitle contribution elicitation wikipedia commoncrawl tatoeba), name, url, license_default, active, last_run_at, rate_limit, proxy_pool, notes
submission_meta	id, corpus_item_id, contributor_name, contributor_email (nullable), self_reported_region, self_reported_origin, session_id, ip_hash, consent_given, consent_voice -- the metadata for one contributed corpus item, named apart from -- "contribution" the act, so the two don't share a word
corrections	id, target_table, target_id, field, old_value, new_value, reason, proposed_by, status, reviewed_by
flags	id, target_type, target_id, reason enum, note, session_id, ip_hash, status enum, resolved_by -- reason: offensive personal_data wrong copyright other -- personal_data reuses the anonymise_revoked routine (§4.3) once -- an admin confirms it – same scrub, triggered by a third-party -- report instead of self-revocation, not a separate manual path
users	id, role, display_name, email, password_hash, trust_level, region_self_reported, active
audit_log	id, actor_id, action, entity_type, entity_id, diff jsonb, ip_hash
stored_objects	id, kind (voice_clip dataset_snapshot db_backup book_scan other), bucket, storage_key UNIQUE, mime_type, size_bytes, checksum, status (pending stored deleted), is_public, expires_at, uploaded_by -- single source of truth for anything in object storage, instead -- of a raw storage_key column on every table that happens to need one
dataset_snapshots	id, version, corpus_item_count, stored_object_id fk UNIQUE, rule_version, notes
model_versions	id, snapshot_id fk, metrics jsonb, trained_at, is_production
standardisation_rules	id, rule_version, pattern, replacement, script, description, active

Every table also carries `created_at` and `updated_at`, timezone-aware. I've left `updated_at` out of the blocks above only so they stay readable — it isn't optional anywhere.

7.9 Fields that are cheap now and impossible later

FIELD	WHY IT HAS TO BE THERE FROM THE FIRST ROW
license rights_status	Scraped and contributed material carry different rights. Without them you can never carve out a commercially clean subset.
dataset_split	Set at insert, never changed. Without it you can't make an honest accuracy claim — you'd be marking your own homework.
rule_version	Spelling rules will change. Snapshots you already published have to stay reproducible after they do.
is_machine	Separates a machine's guess from a human's judgement. Without it, model output contaminates training data and nobody notices.
version	Lets the task engine tell "still valid" from "needs redoing" (§8.1).
consent_voice	Text consent doesn't cover voice, and you can't collect it retroactively (§4.2).

7.10 Publications — phase 2

A page collecting work on Tunisian Derja — papers, theses, corpora, tools — that people can add to and discuss.

```
publications      id, title, authors, year, venue, doi, url,
                  kind enum (paper|thesis|corpus|tool|book),
                  abstract, submitted_by, status enum
                  unique (doi) where doi is not null

publication_comments id, publication_id fk, body, author_name,
                    session_id,          -- same anonymous model as contributions
                    status enum
```

COMMENTS ARE A MODERATION SURFACE NOTHING ELSE HERE CREATES

Everything else in this system is bounded — a region tag, a translation, a vote. Free-text discussion isn't, and it's the one part that can generate work which isn't corpus work. So: same session model as contributions, anything below trust level 1 held for approval, and **the feature gets switched off entirely if it costs more attention than it returns**. Writing that down now means turning it off later is a decision that was always on the table, not a failure.

Seed it before launching it — an empty bibliography reads as abandoned. Prepopulate with the work cited in §1.2 and §13.1: CODA-TUN, NOTA, Tunisiya, TARIC, TSAC, TEC, TunSwitch, TEDxTN, MADAR, MyVoice, DARIJA-C, KALAM'DZ. That reading is done; the page is mostly typing it up.

A researcher landing here should find the field, not just this project. Being where work on Tunisian Derja gets collected is a cheaper way to become a reference point than producing the best corpus — and it turns §1.2 from a defensive paragraph into a living page. It's also the most direct route at problem 4 in §5: people who use a bibliography of Tunisian NLP are people who could review northwest or southern data.

7.11 Media — phase 5, empty for now

These exist in the schema and hold nothing. They're here because the consent question attached to them can't be answered retroactively (§4.2).

```
speakers  id, session_id, region_origin enum, region_residence enum,
          age_band enum, consent_voice bool

media     id, lexicon_variant_id fk, lexicon_form_id fk null, speaker_id fk,
          stored_object_id fk UNIQUE,  -- object storage, not Postgres (§7.8)
          duration_ms, sample_rate, consent_scope enum
```

Voice consent shows up three times across this schema, on purpose, not by accident: `submission_meta.consent_voice` (§7.8) is the opt-in captured at submission time, before any recording exists. `speakers.consent_voice` is that same fact copied forward onto a speaker profile once one gets created in phase 5. `media.consent_scope` is the one that actually governs a specific clip — text-only versus includes-voice — and is authoritative if the other two ever disagree, since it's the closest to the actual recording.

Basic means basic. Phase 5 voice is attestation audio: someone records a clip of a variant they already submitted, and it plays back on the lexicon entry. No ASR, no TTS, no forced alignment, no speech models. The clip proves a written form really is said that way, by someone from there — the one thing text can't prove on its own.

Three things audio changes structurally, which is why it isn't just "a migration later":

1. **Storage leaves Postgres.** A corpus item is ~1KB. A ten-second clip is ~150. That's object storage with a lifecycle policy, and `pg_dump` doesn't cover it — a second backup path with its own economics and restore test (§2.5). The

stored_objects row is resolved through a presigned URL — the clip itself never passes through the API. The provider behind it is pluggable (local disk in dev, R2 or S3 in production, one interface, §10.0) — **built** as control-plane/backend/core/src/storage/, with both a local and an R2 adapter already written, just not wired into a route yet — so this decision doesn't get re-made twice.

2. **speakers has no text equivalent.** A recording attests a variant *as spoken by a person* whose origin may not match the variant's region.
3. **Consent stops being scrubbable.** §4.3 works because a sentence with identifiers stripped is genuinely anonymous. A recording isn't — it's biometric-adjacent and you can't anonymise it by nulling a column. **Voice revocation means deletion**, the second exception to "nothing hard-deletes".

08

The task engine

The piece I'd most regret getting wrong.

Everything below is about curating and verifying material that already exists in the pipeline — reviewing, tagging, translating, adjudicating. It's not the only way in. Anyone can add brand-new material directly, no task required: a new word, sentence or paragraph at /contribute, a new word origin at /origins, a new citation at /research. This section is the review layer sitting on top of that, not a gate in front of it.

Contributing isn't only writing something and walking away, either. Whoever writes a corpus item can optionally translate it themselves, add an Arabic or Arabizi rendering, and say which region it belongs to — standard pan-Tunisian Derja, or a specific region if they know it. Anything they skip doesn't get lost or left half-finished: it becomes exactly the same `translate_*`, `transliterate_*` or `region_tag` task (§8.2) that a scraped corpus item would have spawned. Writing your own material and filling in someone else's are the same task queue, just entered from opposite ends.

ONE STATUS COLUMN CAN'T DO THIS

An entry can be translated but not transliterated, or transliterated but not reviewed. Separate axes of done. A single status column forces them into an order that doesn't exist and makes half-finished work invisible.

8.1 Schema

```
tasks
  id, corpus_item_id fk, type enum
  status    enum    -- open | claimed | done | rejected | needs_rework
  priority  int
  claimed_by fk null, claimed_at, completed_by fk null, completed_at
  output_record_id uuid    -- points at the row it produced
  requires_role enum
  target_regions enum[] null -- the regions in conflict, or the hint (§8.5)
  item_version int
  idempotency_slot text not null default ''
  idem_key   text    -- sha256(corpus_item_id : type : item_version : idempotency_slot)

  index (status, type, priority desc)
  index (claimed_by, status)
  index (corpus_item_id)

  -- at most ONE live task per (corpus item, type, version, slot)
  unique (idem_key) where status in ('open','claimed','needs_rework')
```

WHY THE IDEMPOTENCY KEY EXISTS

"Open task Y if it isn't already there" sounds fine until someone corrects an already-processed corpus item. The correction lands, the corpus item changes, the spawn rules fire again and re-queue `translate_msa` and `link_lemma` — both already finished. Refine another annotation, same thing. The queue fills with duplicates of closed work and reviewers stop trusting their inbox, which is the one surface the whole contribution loop runs through.

- **corpus_items.version** goes up on any accepted change — a correction applied, a standardisation, an adjudicated tag. It's what "this task is about *this* state" means.
- **idem_key** is `sha256(corpus_item_id : type : item_version : idempotency_slot)`. Same four inputs, same key, duplicate rejected by the index instead of queued.

- **idempotency_slot** separates several legitimate tasks of the same type on one corpus item. Empty for item-scoped work (review, region_tag, translate_*, transliterate_*, standardise), char_start:char_end for span-scoped work (link_lemma, adjudicate), an ordinal for the N-of-a-kind case (confirm → 1, 2). Leave it out and the unique index quietly collapses two confirmations into one, one lemma link per sentence instead of one per token, and a second disputed span into nothing. None of which raises an error.
- **The index is partial, over live statuses only.** Putting status inside the unique key instead looks equivalent and isn't — it constrains historical rows too and blocks legitimate rework transitions. Limiting it to open/claimed/needs_rework says the actual intent: never two live tasks for the same work, leave the completed history alone.
- **A version bump is what properly reopens work**, and not every bump invalidates everything. Each rule declares which changes kill its output — a region re-tag doesn't invalidate a finished transliteration; a standardisation does invalidate link_lemma. That mapping sits in the rules module as data.

Built. Schema, atomic claim, complete/reject/rework lifecycle, the idempotency key exactly as above, and the spawn rules for every row in §8.3 — all live in control-plane/backend/core/src/tasks/, a framework-free package the NestJS layer calls into. review, region_tag and confirm completions are wired end to end and have been exercised against a running database. translate_*, transliterate_*, standardise, adjudicate and link_lemma completions aren't wired yet — they need the lexicon and translate modules, which are still empty stubs, and link_lemma additionally needs a tokenisation pass that doesn't exist yet, since nothing currently writes tokens rows.

8.2 Task types

TYPE	PRODUCES	SLOT	MIN ROLE
review	quality + region tag, or a rejection	empty	reviewer
region_tag	region tag	empty	contributor
confirm	agreement vote on an existing tag	ordinal	contributor
translate_msa / _fr / _en	translation row	empty	contributor
transliterate_to_arabic	lexicon form row, script = arabic	empty	contributor
transliterate_to_arabizi	lexicon form row, script = arabizi	empty	contributor
standardise	canonical_form + canonical_map	empty	reviewer
adjudicate	the final call where taggers disagreed	span	reviewer
link_lemma	a links row, token to lexicon variant	span	reviewer

8.3 Spawn rules

```

on corpus item ingested      -> spawn review
on review approved          -> spawn region_tag, translate_msa
on region_tag done          -> spawn confirm slot=1, slot=2 (different annotators)
on translation added         -> if arabizi: spawn transliterate_to_arabic
                             -> if arabic: spawn transliterate_to_arabizi
on consensus agreement < 0.6 -> spawn adjudicate slot=<span> (routed, §8.5)
on corpus item standardised  -> spawn link_lemma slot=<span> per token
on correction accepted       -> bump corpus_item.version
                             -> respawn only what that change invalidates

```

Built, not yet wired. Every rule above exists today as a pure function in control-plane/backend/core/src/tasks/spawn-rules.ts. on review approved, on region_tag done, and the consensus-triggered adjudicate rule are actually called from the running task-

completion code. on translation added and on corpus item standardised are written and tested but nothing calls them yet, because nothing yet creates the translation or standardisation records that would trigger them.

8.4 Claiming and release

- Claiming is atomic — UPDATE ... WHERE status='open' RETURNING. Two workers never get the same row.
- Claims expire; a job releases anything held over two hours.
- One worker holds at most 5 claims at once, so nobody hoards the queue.
- needs_rework isn't rejected: rework goes back in the queue, rejection closes it.

8.5 Regional routing, and the bias it prevents

THE FAILURE I'M DESIGNING AGAINST

When consensus drops below threshold an adjudicate task spawns. If the reviewer pool is northern-heavy — and early on it will be, because that's where my network is — a northern reviewer can confidently reject a perfectly good southern variant just because they've never heard it. That doesn't lose a row. It systematically strips out the exact material this project exists to collect, while looking like quality control the whole time.

So adjudication doesn't go to whoever's free. The task carries target_regions — a set, because §6.4 made region a set and a northwest-versus-south conflict has two regions in it.

```
on spawn adjudicate:
  target_regions = the conflicting tag regions, or the corpus item's document's region_hint

  1. offer ONLY to reviewers whose self-reported region is IN target_regions
  2. after 7 days unclaimed -> open to everyone, flag out_of_region, and say so:
     "no reviewer from this region was available"
  3. out_of_region adjudication can confirm or escalate to admin.
     Never reject on regional grounds, never delete a variant.
```

A reviewer matching any region in the set counts as in-region. The seven-day window is per task, not per region — otherwise a two-region conflict waits twice as long as a one-region one, which is backwards, since disputed cases are the ones that most need deciding.

- Same idea for region_tag and confirm: a confirmation from a reviewer of that region counts at full weight, one from outside counts for less.
- Out-of-region rate is on the admin overview (§17.1). If it climbs, the answer is recruiting in that region — problem 4 in §5 — not relaxing the threshold.
- This is why region_self_reported is on the users table and gets asked at reviewer invitation, rather than being optional profile decoration.

8.6 What this makes easy to answer

What's unfinished by type and age. Which region has the most stalled work. Who's contributing what and how fast. Whether the pipeline is jammed at review, translation or standardisation. All queries, not investigations.

09

Accounts and permissions

Friction kills volume. Anonymity kills quality. Pick per role, not once.

9.1 The tension

Every step between "I want to help" and "I submitted something" costs contributions, and contributions are the resource this whole thing runs on. But annotation quality needs accountability. One policy can't serve both, so there isn't one.

WHY CONTRIBUTORS DON'T GET REAL ACCOUNTS

Somewhere there's a version of this where contributors sign up properly — email, password, verification, a profile. It's cleaner. Every rule gets easier. I'd know exactly who wrote what.

It's also a fantasy, because nobody is doing that. Picture it: someone taps a link, they're curious, they've got maybe fifteen seconds of goodwill, and the first thing they meet is *Create your account*. Gone. Not annoyed, not weighing it up — gone, instantly, the way I'd be gone. I'd be asking a stranger to open an account on a website they've never heard of, run by a guy they've never heard of, so they can donate a word for free.

So contributors get a cookie and a name box, and I live with weaker identity guarantees. That's the whole trade, and it isn't close.

9.2 Tiers

ROLE	AUTH	CAN DO
anonymous	none	browse, search, download, see stats
contributor	name + optional email, no password	submit, confirm and vote, propose corrections, simple tasks
trusted contributor	email verified, trust ≥ 2	+ translation, transliteration
reviewer	full auth, invite only	+ review queue, standardisation, adjudication
admin	full auth + 2FA	+ users, sources, publishing snapshots
superadmin	full auth + 2FA	everything, including destructive operations

9.3 How anonymous contribution works

```
session_id  // httpOnly cookie, uuid, 1 year – ties one person's submissions together
ip_hash     // sha256(ip + server_salt) – abuse detection only, never a raw IP
name        // free text, display only, NOT an identity claim
email       // optional, for credit and follow-up
consent     // explicit checkbox, stored per submission
```

Leaderboard integrity: ranked by *approved* contributions, never raw submissions. Rank by raw and you've just paid people to spam. Name collisions are fine — it's motivation, not a credential. The session id is what actually aggregates someone's work.

9.4 Trust levels

```
trust 0    new / anonymous          -> confirm + submit only
trust 1    3+ approved entries      -> region_tag tasks
trust 2    email verified,
           10+ approved, <10% rejected -> translation, transliteration
trust 3    10+ approved across 2+ task types, <5% rejected
           -> eligible for a reviewer invitation
```

Computed nightly from approval history, not handed out. Trust is earned; the reviewer role is granted. Eligibility never auto-promotes — an admin still sends the invitation, and captures the reviewer's region while doing it, because routing depends on it (§8.5).

9.5 Reviewer login

Full account, not a cookie — email and password, or GitHub OAuth. Every privileged action writes to `audit_log` with actor, diff and ip_hash. The token, session and 2FA design behind this is its own topic, covered fully in §9.7–§9.10 rather than repeated here.

Built so far: email/password only — argon2id hashing, JWT access + refresh with rotation and reuse detection, CSRF double-submit, the full *ThrottlerGuard* → *AuthGuard* → *RolesGuard* → *TrustGuard* chain (§9.6, §9.10). GitHub OAuth isn't wired yet. Registration briefly had a real bug worth recording rather than quietly fixing and forgetting: every signup was hardcoded to `role: 'reviewer'`, so anyone who registered got reviewer access. Found and fixed before this ever left a dev database — it now defaults to `contributor` like it should — but it's the kind of one-line mistake that's worth naming so it doesn't happen twice.

9.6 The hierarchy is internal

Trust levels, roles, task types, spawn rules, consensus thresholds, adjudication routing — all of that is how the system stays honest. None of it is a contributor's problem and none of it appears in front of them.

A contributor sees one box, one question, a button. They don't see their trust level, don't know that finishing a region tag spawned two confirmation tasks, don't pick a task type, and are never told they're ineligible for anything. Work they can't do yet just isn't offered — no locked state, no progress bar toward a permission, no explanation of a system they didn't ask about. Promotion happens silently: more kinds of work start showing up.

The leaderboard is the only visible status anywhere, and it's a ranking rather than a rank — who has contributed, not what tier anyone sits in.

THE RULE FOR EVERY CONTRIBUTOR-FACING SCREEN

One question, three seconds, no explanation required. If a screen needs a paragraph of instructions, the screen is wrong, not the person reading it.

Every sentence between arriving and submitting comes out of a budget measured in seconds. I don't have a citation for how many and I don't need one — I know how fast I close a tab that wants me to read something before it'll let me do anything, and I've got no reason to think anyone is more patient with mine than I am with theirs.

All that complexity stays fully instrumented. It just faces the admin dashboard (§17.1) instead of the contributor.

9.7 Token and session design

JOSE (HS256), not a random JWT library — one well-audited implementation rather than picking whichever package has the most stars this year. A token carries four things beyond the usual `sub` and `expiry`: `token_version` (bumped to invalidate every token for a user at once — the "log out everywhere" and "something's wrong, kill this account's sessions" button in one field), `sid` (session id, so one login can be revoked without touching the others), `rjti` (a rotating token id on the refresh token

specifically, so a stolen refresh token that gets used twice is detectable — reuse means someone else has it). Passwords are argon2id, reviewer- and admin-only since contributors don't have passwords at all (§9.3).

9.8 Session flow

Short-lived access token, longer-lived refresh token, standard pair. Refresh rotates the token on every use — old `rjti` stops working the moment a new one is issued, which is what makes reuse detection possible. Logout revokes the `sid` server-side rather than just deleting a cookie client-side, so a stolen token doesn't stay valid after the legitimate user thinks they've logged out. Bumping `token_version` is the blunt instrument for "everything, everywhere, right now" — used for a password change or a suspected compromise, not for routine logout.

9.9 Email verification and password reset

Both run through a single-use, time-limited OTP sent by email, not a magic link with a long-lived token sitting in someone's inbox indefinitely. Rate-limited per address and per IP, and the response is identical whether or not the address exists in the system — "if an account exists, we've sent a code" — so the endpoint can't be used to check who has an account here.

9.10 Reviewer and admin login specifics

- Failed logins lock the account temporarily after a handful of attempts, with backoff, not a hard permanent lock a legitimate user then has to email me about.
- TOTP 2FA is mandatory for admin and superadmin, optional but encouraged for reviewers.
- GitHub OAuth is preferred over password for reviewers specifically because most of them are likely to be technical people who already have a GitHub account — one less password for me to be responsible for.

DELIBERATELY NOT PORTED FROM ANYWHERE ELSE

A few patterns that show up in other systems I've built or seen and don't belong here, said explicitly so nobody wonders why they're missing: **impersonation** (an admin logging in "as" a user) — nothing here needs it and it's a liability sitting unused.

National ID / QID encryption — this project never collects a government identifier, so there's nothing to encrypt.

Payments — covered already (§4), there is no paid tier, so there's no payment flow to secure. **A separate mobile token flow** — there's no native app (§20), so there's no separate client to design tokens around.

9.11 Email delivery

Outbox pattern: writing "send this email" is a database row in the same transaction as whatever triggered it, not a direct call to an email API in the middle of a request. A worker job dispatches the outbox asynchronously and retries on failure. That means a flaky email provider can never fail the actual action — a signup succeeds even if the verification email is delayed a few minutes, instead of the whole request failing because Resend had a bad second. Resend is primary, with SMTP as a fallback path rather than a second thing to configure and forget about (§2.6).

10

Backend architecture

One monorepo, two apps. Still no ML in the request path — that hasn't changed.

10.0 Technical stack

The full picture before the reasoning. Everything below is a decision, not a placeholder — if it says TBD elsewhere in this document (hosting, domain), that's the exception, not this table.

LAYER	CHOICE	NOTE
Frontend	Next.js (App Router)	SSR for the public pages — §16's citation and search requirements need it
Backend	NestJS 11, TypeScript	Owns the API boundary; guard chain replaces the older single-app route-handler design
ORM	Prisma	Schema in db/
Database	PostgreSQL, behind pgBouncer	Transaction mode, pool 20 (§10.2)
Cache / sessions	Redis	Not the job queue — that's still pg-boss (§10.3)
Job queue	pg-boss on Postgres	Redis-backed BullMQ is a documented future upgrade, not the default
Auth	jose (JOSE), HS256, argon2id	Full design in §9.7–§9.11
NLP service	Python, FastAPI	Health-check only, phase 1 — built , containerised; real model work is phase 4, off-box (§11)
Processing service	Rust, Actix	Health-check only — built , containerised; stays that way until profiling the TypeScript pipeline at real corpus scale shows a specific stage (MinHash/LSH dedup, tokenization, export) is the bottleneck, not on a phase timer alone (§7.11)
Reverse proxy	Caddy (internal)	
Edge / tunnel	Cloudflare Tunnel (cloudflared)	No inbound ports open on the VPS
Email	Resend, SMTP fallback	Outbox pattern (§9.11)
Design system	shadcn/ui + Tailwind	RTL gaps checked, not assumed (§15.2)
Process manager	pm2	Runs the NestJS container in cluster mode, 2 instances, per-instance restart threshold — built , <code>control-plane/backend/ecosystem.config.js</code>
Error tracking	Sentry (@sentry/node, @sentry/react)	Free at this scale — this is how "error rate" on the admin overview (§17.1) gets a source instead of being a number with nothing behind it. Not wired into the backend yet.
Observability	OpenTelemetry + structured logging	A solo-run box with no structured logs is undebuggable the moment something breaks with nobody around to ask. Not wired in yet.
Analytics	PostHog (free tier)	Replaces an earlier guess at self-hosted Umami/Plausible (§20) — reusing a tool already known beats re-evaluating from zero. Not wired in yet.
Object storage	Pluggable: local disk (dev) R2 S3	One interface, provider swapped by env var. Uploads and downloads go via presigned URLs — files never pass through the API (§7.11). Built as an interface plus both adapters (<code>control-plane/backend/core/src/storage/</code>); not called from any route yet.
Realtime	Not default — polling	socket.io + a Redis adapter is the upgrade path if the admin overview or review queue ever genuinely needs push instead of poll (§17.1). Not cut reflexively; just not needed yet.
CI/CD	GitHub Actions	<code>master.yml</code> (lint/typecheck/test/migration-drift/CodeQL, built) and <code>verify-backup-restore.yml</code> (§2.3, built) — <code>infra-watch.yml</code> not built yet
Package manager	pnpm, one workspace, one lockfile	

```

open_derja/      -- ONE repo, ONE pnpm workspace, ONE lockfile
├── control-plane/ -- everything TypeScript: I/O, orchestration, state machines
│   ├── backend/  -- NestJS: guard chain, business logic, data access
│   │   └── core/  -- domain logic: tasks, consensus, trust, standardisation,
│   │               storage. Framework-free, its own workspace package,
│   │               imported by backend, worker, and scrapers
│   ├── worker/   -- pg-boss scheduled jobs; expire_claims live, outbox
│   │               dispatcher not built yet
│   └── scrapers/ -- ingestion runners (bootstrapped as a package,
│   │               pipeline/ and sources/ still empty)
├── data-plane/   -- Rust, Actix — deterministic, CPU-bound batch work.
│   │               Health-check only until a stage proves itself a
│   │               bottleneck at real corpus scale, not on a phase timer
├── intelligence-plane/ -- Python, FastAPI — health-check only, phase 1
│   └── nlp/
├── frontend/     -- Next.js: public site + dashboards
├── shared/       -- types, enums, zod schemas. Zero deps, zero logic.
├── db/           -- prisma schema, migrations, seeds
├── devops/       -- Dockerfiles, compose, Caddy/cloudflared, backups
├── docs/         -- this ledger, blueprint, VOICE.md, CONTRIBUTORS.md,
│   │               verification log
└── testing/      -- e2e, organised by surface

```

The three top-level groups — `control-plane`, `data-plane`, `intelligence-plane` — line up with what actually varies in how each piece runs: TypeScript for anything I/O-bound or a state machine, Rust for deterministic CPU-heavy batch work once it's proven necessary rather than assumed, Python for inference and training kept off the request path. `frontend`, `shared`, `db`, and `devops` sit outside that split on purpose — they aren't plane-specific. `shared` in particular is the vocabulary every plane has to agree on, and since nothing written in Rust or Python can import a TypeScript package anyway, nesting it inside `control-plane` would misstate what it actually is. `core` lives under `control-plane/backend/` rather than at the top of `control-plane/` — still its own framework-free workspace package, just nested one level under the app that uses it most. The dependency direction is unchanged: `backend` → `core` → `db/shared`, `frontend` only ever talks to `backend` through the typed client, nothing imports upward. This is one monorepo end to end, not five separate repos — the whole reason `shared` exists is so a region, a role, or a task type can't mean two different things in two different places, and that only holds if everything sits in one workspace with one lockfile.

nlp/ is Python and can't import shared directly. That's a non-issue for now: neither nlp/ nor the Rust service owns any enum-sensitive logic in their phase-1 health-check-only form — nothing there decides a region, a role, or a task type. Revisit with a generated OpenAPI/JSON contract from shared only if a Python service starts making decisions that need to agree with that vocabulary — most plausibly when phase 4's dialect classifier arrives.

10.1 Shape

Two apps, not one, and still one repo. NestJS owns the API boundary and the guard chain — auth, validation, business logic, data access. Next.js owns rendering — the public site and the dashboards — and talks to NestJS only through a typed client, never straight to the database. The file tree is §10.0; nothing below re-derives it.

Splitting the app didn't change why it's one repo. Enums — region, task type, role, script, origin — are defined once in `shared` and imported by both sides, so a dropdown and a database constraint can't drift apart because they're literally the same file. Zod schemas validate at the boundary and type the forms from one definition. One install, one CI pipeline, one deploy — that part of the reasoning didn't need to change just because the backend got its own process.

Not microservices, and since machine assistance is deferred (§11), not doing any real ML work either — the two service containers in §10.0 exist to prove the deployment shape, not to make decisions. When batch inference eventually happens it runs off-box (§11.3), so it never becomes a third real workload on this box.

10.2 pgBouncer

Sits between the app and Postgres in transaction-pooling mode, pool size 20. Postgres has a hard ceiling on concurrent connections, and NestJS, the worker, and pg-boss all opening their own connections directly is exactly how a small box runs out of them under real load — not a hypothetical, a known failure mode for anyone running Postgres behind more than one process.

One invariant worth stating plainly so it doesn't get rediscovered the hard way: `DATABASE_URL` points at pgBouncer and is what the app uses at runtime. `DIRECT_URL` points straight at Postgres and is what Prisma migrations use — transaction-mode pooling doesn't support the prepared statements and session-level features migrations need. Two URLs, two jobs, and mixing them up is the kind of mistake that only shows up once, at the worst time.

10.3 Background jobs

Redis earns its place for cache and sessions (§9.7), not for the job queue — those are different jobs, and giving them to the same system would mean Redis going down takes both out at once. pg-boss stays on Postgres for the queue, for the same reason it was chosen before Redis was ever in the picture: one less service to run, one less thing to back up, and the volume here never gets near justifying a second queue technology doing the same work Redis could theoretically also do.

JOB	HOW OFTEN	WHAT IT DOES
expire_claims	5 min	release tasks claimed over 2h ago
refresh_consensus	on write + hourly	recompute agreement, flag adjudication
recompute_trust	nightly	update contributor trust levels
refresh_stats	10 min	coverage, leaderboard, translator hit rate
scrape_sources	per-source	pull new material from active sources
anonymise_revoked	hourly	scrub identifiers for revoked consent (§4.3)
export_snapshot	manual / weekly	freeze, checksum, encrypt, upload
backup_db	daily	pg_dump, encrypt, ship off-box

WHAT THAT COSTS, AND THE CAP THAT PAYS FOR IT

pg-boss polls the database. A worker hammering Postgres for spawn, consensus and export work is competing with the same Postgres serving the site — on one small box the queue can starve the frontend. So polling gets capped rather than left on defaults: a 2s floor on the poll interval, a hard ceiling on concurrency, batches small enough that no job sits on a long transaction. Anything bulky — snapshot export, regeneration after a rule bump — runs as one scheduled job at a quiet hour. All config values in the admin panel (§17.2), not constants buried in code.

10.4 API conventions

POST	/api/corpus-items/contribute	create (contribution) – BUILT
GET	/api/corpus-items/:id	entry permalink data – BUILT
GET	/api/corpus-items?region=&...	browse / search
POST	/api/tasks/claim	atomic claim – BUILT
POST	/api/tasks/:id/complete	submit result – BUILT
POST	/api/tasks/:id/review	review-task completion – BUILT
POST	/api/tasks/:id/region-tag	region_tag completion – BUILT
POST	/api/tasks/:id/confirm	confirm completion – BUILT
POST	/api/tasks/:id/skip	release, no penalty
GET	/api/tasks/inbox	what this user can work on
POST	/api/tags	add a tag
POST	/api/corrections	propose correction
POST	/api/flags	report something (§16.4)
GET	/api/translate	the lookup endpoint (§14)
GET	/api/stats/coverage	public coverage data

The contract itself hasn't changed — only which framework serves it has. Every route above is the same route it was when a single Next.js app handled it; NestJS just answers now. The BUILT markers are the actual state as of this revision — everything else on the list is still the plan, not the code.

11

Machine assistance — deferred

Planned, scoped, and not built from day one.

THE DECISION

There's **no machine learning in this system at launch** — no model, no inference, nothing in the request path that can be slow, wrong, or down. There is a Python service from phase 1, deliberately minimal: a health-checked FastAPI container with no model loaded and no meaningful memory footprint, existing solely so the deployment shape — a container, reachable, health-checked, sitting in the compose file — is proven early instead of showing up as a phase-4 surprise. The distinction that matters is unchanged: the service exists, but nothing in it makes a decision yet. Dialect classification, embeddings and machine translation are still **phase 4** — and when they show up they run off-box, not on the application server.

11.1 Why it's deferred

- **It was the biggest thing driving infrastructure cost.** A MARBERT-class model sits on 1.5–2GB. Even loading it only at night, firing it up while Postgres is chewing through the queue is a realistic way to get OOM-killed. Dropping it doesn't manage that risk, it deletes it (§2).
- **It doesn't do the interesting part.** There's no pretrained model at Tunisian sub-region level. A classifier can tell you a sentence is Tunisian. It can't tell you it's from Kef and not Sfax. That call — the actual contribution here — is human either way.
- **You can't train it before the corpus exists.** Regional classification needs this dataset first. Building the consumer before the input is backwards.

11.2 What does the job instead

Rules: script detection, length bounds, a regex marker pre-filter, near-duplicate removal (§12.3). Weaker, and §12.3 says exactly what that costs. I'll take it over needing a real model and a GPU on day one — the sidecar already exists (§10.0), it just isn't doing anything yet. Transliteration is unaffected — that was always a rule engine (§13).

11.3 What phase 4 might look like

Not planned in any real detail — this is phase 4, and I'll design it properly when I get there. Just so it's not a total blank, roughly what I'd reach for:

- A dialect pre-filter — something MARBERT-class, run offline over the backlog to cut reviewer load, not to make decisions.
- A regional classifier, trained on this corpus once it's big enough — the actual research contribution, and the reason the corpus has to come first.
- Hosted embeddings + pgvector for fuzzy matching. Exact and trigram lookup already ship in phase 2 with no model at all (§14.2), so this only covers the fuzzy case.

Whatever it ends up being, it runs off-box, billed by the second, not on the application server — that part I'm sure of, since it's what keeps the box small (§2). And whenever machine output does arrive it enters as a suggestion, `is_machine: true`, never straight into a canonical field (§7.9, §12.4). That rule doesn't wait on the details above being settled.

Ingestion and scraping

Automation exists to protect the one thing I can't buy more of: reviewer attention.

YOUTUBE IS NOT A SOURCE

The YouTube API Developer Policies require comment text, video titles and creator names to be refreshed or deleted within 30 days of storage — a rolling clock, permanent, for as long as any of it is held. That's incompatible with a corpus built to be permanent, and it directly contradicts the invariant that `raw_body` is immutable (§7.2, §12.6). YouTube comments are excluded as a source. Forums are the phase 1 source instead.

12.1 Source types

SOURCE	YIELD	DIFFICULTY	RIGHTS
Forums	high volume, informal, script-mixed	medium — HTML parsing	No platform-mandated retention clause. The phase 1 source. Each site's ToS read before it goes live (§12.2).
Direct contribution	low volume, best quality	none	Clean — explicit consent. The only fully commercial-safe tier.
Elicitation	aimed at known gaps	none	Clean.
Books / print	low volume, high quality, historical	high — OCR, RTL, layout	Copyright unless public domain. The main route to era: historical.
Subtitles	medium, lots of dialogue	medium	Usually copyrighted. Per-source licence check.

SCRAPING MAKES THE EXACT BIAS THIS PROJECT EXISTS TO FIX WORSE

What's online is disproportionately northern and urban. Scrape only, and you get a Tunis-skewed corpus — guaranteed. The counterweight is **targeted elicitation**: the system finds lexicon entries with no southern variant, or sentences with no northwest equivalent, and asks people from those regions directly. A gap becomes a question someone can answer in ten seconds instead of a vague plea for help.

12.2 What gets scraped and what doesn't

SOURCE	STATUS	WHY
Tunisian forums	In use	HTML fetch under each site's own ToS, not a platform API contract with a retention clause attached. Each site's ToS is read before it goes live.
Blogs, personal sites	In use	Same terms as forums. Ordinary copyright applies to what's stored (§12.6).
News site comments	Case by case	Per-site ToS read, same process as forums.
Tunisian Arabic Wikipedia (aeb.wikipedia.org), Wiktionary	In use	CC BY-SA, explicit dumps published for reuse, no refresh requirement, no rate-limit risk. Small but already segmented and already written Derja in Arabic script — good for validating the pipeline against before fighting a platform API.
Common Crawl, filtered to .tn domains	In use	Permissively licensed for exactly this, enormous, no platform relationship to maintain.
Tatoeba	In use	CC-BY sentence pairs, some Tunisian Arabic, already parallel — directly useful to the translator.
YouTube (API)	Excluded	The 30-day refresh/delete rule (§12.1) is incompatible with a permanent corpus and directly contradicts raw_body being immutable. Settled, not a "for now" — full stop.
Facebook / Instagram	Excluded	ToS forbids automated collection; Meta litigates against scrapers.
Reddit	Excluded	API is paid with its own retention and commercial-use terms since 2023.
Twitter/X (API)	Excluded	Paid, restrictive storage and redistribution terms.
TikTok	Excluded	Own developer API with commercial terms; not worth the effort at this scale.

The three additions since the first draft of this document — Wikipedia, Common Crawl, Tatoeba — are what actually made it into the phase 1 scraper list once I looked for open, dump-based sources instead of only platform-shaped ones. The licence-travels-with-the-data argument below is why they beat the platform APIs on the excluded side, not just permission on paper.

RIGHTS POLICY FOR SCRAPED TEXT

Every source is checked against its terms of service, not just its robots.txt — the two are separate documents and permission from one doesn't imply the other. Scraped and stored text is treated as research-use only and kept out of the commercially clean export tier by default (§4.4), regardless of the source. Data protection follows the GDPR posture in §4, since hosting sits in the EU.

Why open dumps beat platform APIs, stated once so I don't forget it. Licence has to travel with the data. A CC-BY-SA dump downloaded once is mine to keep and republish indefinitely. Anything accessed through a platform API carries that platform's leash forever — refresh obligations, revocable authorisation, terms that can change under me with 30 days' notice. A corpus meant to outlive my attention can only be reliably built from the first kind, which is the real reason Wikipedia, Common Crawl and Tatoeba are in and YouTube, Reddit and Twitter/X are out — not just that the APIs cost money.

The filter logic that decides what a scraper keeps versus throws away isn't decided yet either — what's below is the shape of it, not a final ruleset. Expect the actual thresholds and markers to move once real data starts coming through.

12.3 Pipeline stages

```
raw fetch
-> store verbatim in documents.raw_body (never touched again)
-> clean (strip urls, mentions, emoji, normalise whitespace)
-> length filter (3–500 tokens)
-> script detect
-> regex marker pre-filter (Tunisian-specific lexical markers)
-> near-duplicate removal (MinHash LSH, threshold 0.85)
-> split into corpus items (paragraph -> sentence)
-> compute match_key (§13.5)
-> insert corpus items, assign dataset_split
-> spawn review tasks

[phase 4] dialect classify – offline batch, slots in before the split into corpus items
```

REVIEWER QUEUE FATIGUE — WHAT DROPPING THE CLASSIFIER COSTS

Rules alone let more noise through than a model would. Best estimate: dropping a machine-learning dialect filter raises noise leakage — MSA and other Maghrebi dialects that aren't actually Tunisian Derja — from somewhere around 20–30% up to 40–50% of what reaches the queue. That's a real, noticeably heavier manual review load through phases 1–3, not a rounding error.

The fix is throughput, not cleverness: small ingest batches, the marker list kept as versioned data so it improves from real rejections, and a source's reject rate treated as a reason to switch it off. Whether it's worth pulling a lightweight dialect filter earlier than phase 4 to cut that leakage sooner is a real question I haven't settled — noted here rather than decided.

SCRAPING DOESN'T SERVE THE REGIONAL-SIGNAL ARGUMENT — A REAL LIMIT, NOT A FOOTNOTE

A scraped sentence has no region metadata. None. So how does a reviewer tag it *northwest*? By recognising northwestern markers in the text — which means the regional label on scraped material encodes what reviewers already believe is regional, not independent evidence that it actually is. That's circular. Feed that into a classifier later (§11.3) and I've trained a model to reproduce my reviewers' priors and called it a finding.

Contributed material is different in kind: someone says "I'm from Kef, we say it like this." Weak evidence, but *independent* of the linguistic features I'm trying to learn — testimony, not inference. **Consequence, as policy:** scraped corpus items default to **region: unknown** rather than being tagged from linguistic intuition. A reviewer may tag a scraped item's region only when a contributor from that region has independently attested the same variant. Scraping buys volume and general Derja coverage. It cannot, on its own, buy regional claims — elicitation is the load-bearing pipeline for what this project is actually differentiated on.

12.4 Dedup applies to contributions too

MinHash sits in the scraper pipeline, which leaves an obvious hole: someone submits a sentence already in the corpus, and it sails through as new work for a reviewer to reject.

So the same near-duplicate check runs on the contribution path, against `match_key` — and the response is the interesting part. Not "rejected, duplicate", which reads as a slap. Instead: *we've already got this one — but nobody has recorded how the south says it. Do you know?* That turns a rejection into an elicitation prompt, which is the same trick §14.4 plays on the translator's misses.

12.5 Staying inside the rules, not just avoiding a ban

Two different problems live here and they need two different fixes. Getting IP-banned is an operational annoyance. Violating a platform's terms of service, or a data protection law, is a different kind of problem — the fix isn't a proxy, it's not scraping that source at all (§12.2).

- **Every source gets its ToS read before its first run**, not just its robots.txt checked. The two are different documents answering different questions, and a site can allow crawlers in one while forbidding storage in the other.
- **One server means one IPv4**, so scraping at volume from a single address risks a ban even on a fully permitted source. Budget for proxy rotation on anything HTML-parsed — planned for up front rather than discovered the week after the first ban.
- **Rate limits and backoff are per-source config**, on the source row, editable from the admin panel, exponential backoff on 429 and 403.
- **Two failed runs disables the source** and raises an alert (§17.1), instead of continuing to hammer a host that already said no.
- **Scrape slowly and constantly** rather than in bursts. There's no deadline — this is supposed to accrue for years, and being polite is both the compliant choice and the operationally cheaper one.

12.6 Rules

- Raw material is immutable. Every stage reads one representation and writes a new one — which is exactly why a source with a mandatory refresh/delete clause (§12.1) can't be a source here.
- Machine output — whenever it exists — comes in as a suggestion, never as fact.
- Every source is registered with its default licence *and* its ToS status from §12.2, so rights attach at ingest instead of being guessed later.
- Scrapers respect robots.txt, rate limits, and the site's terms of service — the first two are not a substitute for the third.
- Author identifiers stripped at ingest. No personal data from social sources, ever (§4.4).

Standardisation, transliteration, matching

Three different jobs on the same text. Keeping them apart is the whole trick.

13.1 Don't invent an orthography

CODA-TUN (LREC 2014) and NOTA already define conventional spelling for Tunisian Arabic, and CODA* maintains a verified spelling database across dialects. Using them beats inventing something: peer-reviewed, free, and it makes this dataset work with the whole CAMEL and MADAR toolchain. Inventing a rival standard costs months and makes the data less useful at the end of it.

13.2 Rules are versioned data, not code

```
standardisation_rules
  id, rule_version, pattern, replacement, script, description, active
```

A spelling rule will change six months in. If canonical forms get overwritten in place with no version recorded, every snapshot you've already published becomes irreproducible and internally inconsistent. Rules as versioned rows make regeneration deterministic — and re-runnable at an older version, which is what makes a bad rule bump recoverable (§17.2).

13.3 Arabizi mapping

Arabizi to Arabic isn't one-to-one. Digits stand in for letters with no Latin equivalent, and several mappings depend on context.

ARABIZI	ARABIC	ARABIZI	ARABIC	ARABIZI	ARABIC
2	ء	3	ع	5	خ
6	ط	7	ح	8	غ
9	ق				

AMBIGUOUS	COULD BE	NOTE
"e"	ي / ه	or silent
"a"	/ ة	or a fatha
"ch"	ش	French influence — not "sh"
"ou"	و	French influence
"th"	ث / ذ	depends on context

This table is worth publishing on its own — useful to people who don't care about the rest of the corpus, and a cheap way to look like I know what I'm doing early on.

13.4 The standardisation workflow

1. The rule engine produces a suggested canonical form and marks the spans it wasn't sure about.
2. The reviewer sees the suggestion with those spans highlighted and fixes them inline.
3. The accepted form is stored with its `rule_version` and with `canonical_map` — the offset alignment between original and canonical form, which the engine already computed while applying its rules. That map keeps token spans (§7.2) usable

against both versions of the text.

- Corrections that keep recurring are candidate rules. The reviewer UI surfaces the manual overrides people make most often.

13.5 The match key — a third representation

POSTGRES HAS NO ARABIC TEXT SEARCH CONFIGURATION

This matters and it's easy to miss. Postgres ships text search configs for English, French, Spanish, Turkish and a dozen more. There is no Arabic one. `to_tsvector('arabic', ...)` doesn't exist. Any plan that says "we'll use full-text search" for the translator is writing a cheque the database won't cash.

Three options, and this document picks one rather than leaving it open:

OPTION	VERDICT
simple config + own normalisation	Works. No stemming, so كتب won't match يكتب . For lexicon lookup that's mostly fine — I'm matching attested forms, not inflected free text.
pg_trgm	Character trigrams, language-agnostic, genuinely good at typo tolerance and spelling variation — which is most of what Derja search actually is. The workhorse.
Snowball Arabic stemmer	Exists, tuned for MSA. MSA stemming applied to Derja will be wrong often enough to hurt. No.

Decision: pg_trgm plus the simple config, no stemmer. Derja has no standard morphology to stem against, and I'm matching against canonical forms I control.

Normalising before anything is compared

Without this the translator misses constantly, because Arabic has several ways to write the same thing and people use all of them. Before indexing or comparing:

alef variants collapsed	<- ا آ إ
taa marbuta -> haa	ة <- ه
alef maqsura -> yaa	ي <- ي
diacritics stripped	fatha, damma, kasra, shadda, sukun
tatweel removed	—
Arabic-Indic digits	٩-٠ -> 0-9 (Arabizi mixes both)

THIS IS NOT THE SAME JOB AS CODA STANDARDISATION

CODA produces the canonical form that gets *stored and displayed*. This produces a match key that nobody ever sees. Conflating them is a real trap: you want search to be aggressive and forgiving, and you absolutely do not want that aggressiveness leaking into your published canonical forms. Two layers, two purposes, two columns.

```
corpus_items.match_key    text    -- normalised, indexed, never displayed
lexicon_variants.match_key text
lexicon_forms.match_key   text
index gin (match_key gin_trgm_ops)
```

14

The translator

The reason anyone opens the site. Ships in phase 2, needs no model.

14.1 What it is

- **Derja** → **MSA / French / English**. Any script, any region.
- **MSA / French / English** → **Derja, with a region selector**. Nobody else offers this direction. "Say this in southern Derja" is something no existing tool can do, and it's the demo that makes people send the link on.

Output isn't one string. It's the attested forms with their regions, their scripts, a link to the corpus item each came from, and — where anyone recorded it — where the word came from (§7.6). That last detail costs nothing and does a lot of work: seeing that a word you use every day arrived through Ottoman Turkish is what makes the page feel like a reference work instead of a lookup table.

14.2 It needs no model

- **Word and phrase** — lookup through `lexicon_entries` → `lexicon_variants` → `lexicon_forms`. Exact match on `match_key`, then normalised match through the standardisation rules, then `pg_trgm` similarity for typos and spelling variation (§13.5).
- **Sentence** — trigram similarity over `corpus_items.match_key`, returning attested corpus items with their human translations. Not full-text search, for the reason in §13.5.
- **Region filtering** — a `lexicon_variant_regions` join. Already in the schema.

No embeddings, no API key, no cost, nothing in the request path that can be down.

14.3 Why it improves on its own

THE PROPERTY THAT MAKES THIS THE RIGHT SHAPE

Every accepted annotation improves the translator the moment it's written. No retraining, no snapshot to promote, no regression risk. A trained model improves in versioned jumps and can get worse; a retrieval translator can only get better, and it gets better the same day someone contributes. That's worth more here than fluency — it makes the tool a live reason to contribute instead of a thing that consumes contributions.

The page says so, with a live counter: *this translator knows N words and M sentences across four regions, up from X last month*. On day one that counter says something embarrassingly small and I'm leaving it visible anyway, because "12 words, growing" is a more honest pitch than a splash page pretending to be finished.

14.4 Failing honestly, and the loop that comes out of it

When there's no attestation, the translator doesn't guess. It shows which words it found and which it didn't, the nearest attested neighbours marked as approximate, what's missing by region — *"attested in north and sahel. No southern attestation."* — and a one-click contribute prompt for exactly that gap.

EVERY FAILED LOOKUP IS AN ELICITATION PROMPT

The translator's gaps *are* the `/gaps` page, generated from real demand instead of guessed at. Someone from the south who searches for a word and sees "nobody has recorded how the south says this" is the best contribution prompt in the system, and it costs nothing to produce.

14.5 Staging

PHASE	CAPABILITY	NEEDS
2	Word and phrase lookup, exact + normalised + trigram, region-filtered	lexicon + pg_trgm
2	Sentence retrieval by trigram similarity	match_key + GIN index
3	Reverse direction with region selector	lexicon coverage across all four regions
4	Fuzzy semantic matching	hosted embeddings + pgvector
5	Grounded generation for unattested input	corpus + API, labelled as generated

14.6 What it isn't

The page says what it is: a corpus lookup, not machine translation. It returns what Tunisians have actually written, or it returns nothing. That's a weaker product than a fluent model and a more honest one, and the honesty is why a researcher can cite it.

The same caution covers anything generated. A model trained only on this corpus will be bad for a long time, and putting one in front of people early burns credibility with exactly the audience I need. Generation is phase 5, labelled, never mixed in with attested results. When models arrive they train against frozen snapshots and get promoted only on measured improvement against a held-out split that never enters training — no continuous retraining, because that gives no reproducibility, no way to spot a regression, and quietly absorbs bad annotations.

WHAT THIS COSTS: THE LEXICON LANDS IN PHASE 2

lexicon_entries, lexicon_variants, lexicon_variant_regions, lexicon_forms and lemma-linking are what the translator runs on, so they belong in phase 2 rather than with the rest of the service layer. Lemma linking is core annotation work here, not a late nicety.

Frontend and design system

It's a language project. The interface had better be good about language.

15.1 Stack

CONCERN	CHOICE	WHY
Framework	Next.js App Router	Public pages need SSR for search and citation; dashboards are client-heavy. One framework covers both.
Styling	Tailwind + CSS variables for tokens	RTL through logical properties, not direction hacks.
Data fetching	Server components public, TanStack Query dashboards	Dashboards need optimistic updates and background refetch. Public pages don't.
Forms	react-hook-form + zod (shared schemas)	Same validation both sides, one definition.
Tables	TanStack Table	Review queues need virtualisation, sorting, column control.

15.2 Bilingual and bidirectional from the start

Arabic content sits inside an English or French interface constantly. Mixed direction is the normal case here, and retrofitting it costs far more than doing it first.

- CSS logical properties everywhere — `margin-inline-start`, never `margin-left`.
- Every Derja text node gets `dir="rtl"` and `lang="ar-TN"`, whatever the surrounding UI is doing.
- LRM/RLM marks around mixed-script inline content, or the bidi algorithm reorders it into nonsense.
- A typeface with real Arabic coverage. System defaults render Tunisian text badly.
- Arabizi is Latin script but Tunisian language: `lang="aeb-Latn"`, LTR. Neither of the other two cases.

Three UI locales: English, French, Arabic. French matters more than it looks — a lot of Tunisian users read interface text more comfortably in French than in MSA. Locale is independent of content direction.

15.3 Annotation components

The dashboards share a small set of components. Building these properly once is most of the frontend work.

COMPONENT	WHAT IT DOES
<CorpusItemText>	Correct dir/lang, highlights tagged/linked spans, select to create new ones
<SpanSelector>	Mouse and touch selection returning character offsets, snapping to token boundaries
<TagChip>	Kind + value, colour-coded, removable, annotator on hover
<RegionPicker>	Four regions, multi-select (§6.4), plus pan-Tunisian and unclear. Keys 1–5.
<TaskCard>	Wraps any task type, handles claim/complete/skip the same for all
<ScriptToggle>	Switches a corpus item between Arabic and Arabizi views
<DiffView>	Original against standardised, character-level highlighting

Built so far: none of these seven components exist yet. The 31 frontend routes in §16 are bare stubs — a heading and one line of description each, enough for `next build` to compile and every route to return 200, nothing wired to real data. This whole component set is still ahead.

15.4 Keyboard-first for reviewers

Someone working through hundreds of entries in a sitting is limited by mouse travel. Full keyboard operation is a throughput feature, not an accessibility checkbox, and it gets designed in.

1-5	set region	a	approve
j / k	next / previous	r	reject
enter	submit + next	s	skip
t	focus translation	e	edit standardised form
?	shortcut help	f	flag for adjudication

16

Every page, specified

With the phase attached, so nothing gets built early.

These are the basic screens as I understand the need for them right now. I expect to find pages that need to be added, split, merged, or dropped once I'm actually building and using them — this list is a starting point, not a locked spec.

16.1 Public pages

PAGE	ROUTE	WHAT'S ON IT	PHASE
Landing	/	What this is, live counts by region, a compare example, contribute CTA	2
Translate	/translate	Live translator, both directions, region-aware, honest about what it hasn't got (§14)	2
Explore	/explore	Search and filters: region, scope, era, setting, register, unit, script, has-translation	2
Compare	/compare	One lexicon entry, four regional columns. The page people share.	2
Entry permalink	/s/:id	Text, tags, translations, attestations, source, revision history, cite button, report link (§16.4)	2
Lexicon	/lexicon	Lexicon entry list, regional variants, attestation counts	2
Lexicon entry	/lexicon/:id	One lexicon entry, every variant and form, example attestations, origin with derivation note (§7.6)	2
Origins	/origins	Browse by layer — every Amazigh-derived word, every Italian borrowing, the -j i suffix note — and add an origin yourself if you know one nobody's recorded (§7.6)	2
Research	/research	Bibliography of work on Tunisian Derja. Community-submitted, seeded before launch (§7.10)	2
Download	/data	Versioned releases, formats, checksums, licence per tier, changelog	2
Methodology	/methodology	Sourcing, CODA adoption, annotation process, agreement rates, known biases	2
Coverage	/coverage	Regional balance, unit distribution, translation completeness, gaps	2
Style guide	/style	Orthographic conventions, the Arabizi mapping table, worked examples	2
Citation	/cite	BibTeX, Zenodo DOI, version-specific citation strings	2
Licence	/license	Per-source rights, which tiers are commercially usable	2
Gaps	/gaps	Most wanted, driven by translator misses and coverage deficits	3
Leaderboard	/contributors	By approved count, and by region	3
Support	/support	Donations and sponsorship only — nothing sold, nothing gated behind it. What the money would go toward.	3
API docs	/docs/api	Endpoints, auth, rate limits, examples	4

16.2 Contributor pages — one question, three seconds

Every one of these is built against §9.6: none of them exposes trust levels, task types, or any part of the machinery behind them.

PAGE	ROUTE	WHAT'S ON IT	PHASE
Contribute	<code>/contribute</code>	One box: write a word, a sentence, or a paragraph. Region as four tappable buttons, consent as one checkbox, no account. Translation, script and region detail are all optional extras below the fold — skip them and someone else's task queue picks up exactly what's missing (§8).	3
Confirm	<code>/confirm</code>	One item, one question, three buttons, nothing else on the page. The lowest-effort surface there is — this is what turns a visitor into a contributor.	3
Task inbox	<code>/tasks</code>	Open tasks this person can do. Never shows an empty or locked state — if there's nothing, it redirects to <code>/contribute</code> .	3
Translate task	<code>/tasks/translate</code>	Corpus item plus draft, fix and submit	3
Transliterate	<code>/tasks/transliterate</code>	Corpus item plus rule-generated suggestion, uncertain spans highlighted	3
Propose correction	<code>/s/:id/correct</code>	Suggest a fix without write access	3
My contributions	<code>/me</code>	What they contributed and whether it was accepted. Not a level, not a tier, not a path.	3

16.3 Reviewer pages — logged in

PAGE	ROUTE	WHAT'S ON IT	PHASE
Review queue	<code>/review</code>	Triage: approve, reject, region-tag, quality-tag. Keyboard driven.	1
Lemma linking	<code>/review/lemma</code>	Attach tokens to lexicon variants. Core annotation work — the translator runs on it.	2
Adjudication	<code>/review/adjudicate</code>	Conflicting tags side by side with who made them; pick or override. Region-routed (§8.5).	3
Corrections	<code>/review/corrections</code>	Community fixes, origin proposals, publication submissions, flags	3
Standardisation	<code>/review/standardise</code>	Suggestion against original, uncertain spans, accept or edit	4

16.4 Two things public pages have to decide

Reporting

§4.4 promises removal requests get honoured quickly, which is hard to do with no way to make one. Every permalink gets a report link writing to `flags` (§7.8), feeding the corrections lane. I will get slurs, insults, someone's phone number, and material a person wants gone — that's not pessimism, it's what happens to every site that accepts public text. A button is cheaper than an inbox.

Vulgar register

register: vulgar exists because Derja is full of it and excluding it would falsify the corpus. But `/explore` and `/translate` are public and unauthenticated, so the policy has to be stated rather than left to whoever writes the query:

- **Filtered by default, with a visible opt-in toggle.** The toggle is honest about what it does — it doesn't say "safe mode", it says "include vulgar register". The preference lives in a cookie, same idea as the session cookie in §9.3 — no account needed to keep the choice between visits.
- **The API carries the same switch, not a separate policy.** `/v1/search` and `/v1/lookup` both take an `include_vulgar` parameter (§18.1) that does exactly what the UI toggle does. Without it, both default to filtered — the API doesn't get a quieter or looser rule than the page does.

- **Always present in exports**, always labelled. A corpus that quietly drops half of how people actually speak isn't a corpus of how people speak.
- **Never filtered in the review queue**. Reviewers see everything; that's the job.

17

The admin dashboard

This page decides whether one person can actually run this thing.

The entire audience for this page is me, at odd hours, probably slightly panicked. Everything on it earns its place by that standard.

17.1 Overview — the one screen that answers "is it okay"

WIDGET	SHOWS	ALERTS WHEN
Pipeline throughput	ingested / reviewed / published, 24h and 7d	review rate under ingest rate for 3 days
Task queue health	open by type, oldest open, average time to complete	anything older than 14 days
Regional balance	live share per region against target	any region under 15% of total
Annotator agreement	rolling agreement rate, adjudication backlog	agreement under 0.7
Out-of-region adjudication	share decided by a reviewer from elsewhere (§8.5)	over 20% for any region — a recruiting problem, not a threshold to loosen
Translator coverage	hit rate on lookups, miss rate by region	any region's miss rate over 60% — the gap the corpus is failing at, measured by real demand
Contributor activity	active contributors 7d, new against returning	no new contributors in 7 days
Scraper status	per-source last run, yield, error rate	any source failing twice in a row
Service health	nlp and media container status, uptime (backend itself runs under pm2, §10.0)	either down more than 5 minutes
System	DB size, disk, memory, queue depth, error rate, last backup	disk over 80%, queue depth over 1000, backup older than 48h

Every "error rate" on this screen is backed by Sentry (§10.0) — DSN configured, default trace sampling — not a number invented for the dashboard.

This screen holds every number §9.6 keeps away from contributors. That's the point — the complexity is real and fully instrumented, it just faces me instead of them.

17.2 Sections

Data management

- Browse every table with filters — a safe read-only view
- Bulk operations: re-run standardisation on a selection, reassign region, bulk reject a bad scrape batch
- Merge duplicate lexicon entries; split a lexicon entry that conflated two meanings
- Every bulk operation writes an `audit_log` row with a reversible diff
- Translator miss log — most-searched unattested terms, ranked, exportable as elicitation prompts

Review lanes

- Community corrections, proposed word origins, submitted publications and flags all ride the one `corrections` lane rather than getting queues of their own — flags get a bare list view from phase 2 (§20), folded properly into this unified lane

once the rest of it exists in phase 3

- Publication comments awaiting approval, and the switch that turns commenting off entirely (§7.10)

Task administration

- Reprioritise queues — push southern-region tasks up when balance slips
- Force-release stale claims, reassign, manually spawn tasks for a selection
- Pause a task type entirely — suspend translation while standardisation catches up
- Adjudication routing: review out-of-region cases, adjust the 7-day window, see the rate per region
- Queue throttle: poll interval, concurrency, batch size, live (§10.3)

Sources

- Add, edit, disable; per-source default licence and consent status
- Schedule, rate limit, proxy pool, yield history
- Manual run; last error with stack trace
- Quarantine a source and bulk-flag everything it produced, if the rights turn out murky
- Source failures and the nlp/media health checks (§17.1's Service health row) alert through the same Sentry path — one notification mechanism, not two

Users and contributors

- Promote to reviewer; adjust trust manually with a recorded reason
- Per-contributor quality: approval rate, disagreement rate, regions they submit from
- Ban by session or ip_hash; bulk-revert everything a bad actor submitted
- Invite reviewers by email with expiring tokens, capturing their region

Standardisation rules

- Full CRUD, with a preview showing how many corpus items a change would touch *before* applying it
- Bump rule_version, then regenerate canonical forms and offset maps as a background job
- Frequent-override report: the corrections reviewers keep making by hand — the candidate new rules

A BAD RULE BUMP NEEDS A WAY BACK

Regenerating every canonical form from new rules is exactly the operation you discover was wrong afterwards. So: **snapshot before regenerating**, keep the prior rule_version rows active rather than deleting them, and make regeneration re-runnable at any earlier version. Rolling back is then re-running an old version, not restoring a backup at 3am.

Snapshots and releases

- Create: freeze, checksum, count, release notes
- Pick export tiers — full, commercially clean, etymology, per-region
- Push to Hugging Face and object storage via presigned URLs, never through the NestJS process — same mechanism §7.11's media storage will need later
- Diff two snapshots: corpus items added, changed, removed; rule version delta
- Deprecate with a reason, shown publicly on the download page

Privacy operations

- Process a consent revocation with typed confirmation (§4.3)
- Preview which fields in which tables a revocation will scrub, before running it
- Subject access request: export everything held about one session or account
- Revocation log — action, timestamp, count, and no identifiers in it

System

- Job status, failure log, manual retry
- Backup status: last successful dump, size, destination, encryption confirmed, last restore test — populated by `verify-backup-restore.yml` (§2.3), not a calendar reminder to check manually
- Feature flags — turn a page on for reviewers before it goes public
- Maintenance mode with a public banner
- Full audit log, searchable by actor, entity, action, date

Model management — phase 4 only

Register a model against a snapshot with its metrics; compare a candidate against production on the held-out split; promote or roll back. Empty until §11 happens — specified now so the frozen-split discipline is designed in rather than bolted on.

RULES FOR ANYTHING DESTRUCTIVE

Nothing hard-deletes. Every destructive action is a soft delete plus an audit entry. Bulk operations over a threshold make you type the affected count to confirm. This dataset is the whole asset — the admin panel should make it hard to damage even when I'm tired and moving fast.

18

The public API

Convenience and lookup, free at every tier. Nothing here is ever sold.

18.1 Endpoints

The whole API is phase 4. Within it, the rule-based endpoints work as soon as it exists; the two that need a model wait on §11.

```
GET /v1/search          q, region, scope, unit, script, limit, include_vulgar
GET /v1/corpus-items/:id full record with tags, tokens, links, and translations
POST /v1/normalise      { text } -> { canonical, script, rule_version }
POST /v1/transliterate  { text, from, to } -> { text, ambiguous[] }
POST /v1/lookup         { text, region?, include_vulgar? } -> attested forms + translations
                        exact / normalised / trigram. No ML. (§14.2)

GET /v1/lexicon/:entry_id all regional variants
GET /v1/origins/:entry_id etymology, where recorded
GET /v1/coverage        public statistics

    waiting on the deferred model work (§11.3):
POST /v1/classify       { text } -> { regions[], confidence, alternatives[] }
POST /v1/memory         { text } -> fuzzy semantic matches; needs embeddings
```

18.2 Access tiers

All free. Tiers exist to keep the API usable under real traffic, not to charge for it.

TIER	LIMITS	FOR
Anonymous	60 requests/hour, search and corpus-item read only	having a look
Free key	1,000 requests/day, all read endpoints	researchers, hobby projects, tool builders
Research / heavy use	unlimited, on request	academic use, institutional partners, anyone with a real reason for more than the free tier gives — granted by hand, still free

The numbers above are a starting guess, not a measured limit. Real rate limits and download caps depend on what the VPS can actually handle, and I can't say that honestly before hosting is decided (§2.1). These get tuned against the real box once it exists.

18.3 Rules

- **There is no paid tier and there never will be.** The API is not a revenue plan wearing a rate limiter. This is a non-profit effort — tiers exist to protect the server from getting hammered, not to monetise access. A company wanting to build a paid product on top of this data is free to; I'm not the one charging them for the privilege.
- **The bulk corpus is never gated.** That openness earns the goodwill everything else depends on.
- Every response carries the `snapshot_version` and `rule_version` it was computed against.
- Rate limiting at the reverse proxy, not in application code.

Export and publication

The formats each audience already uses, and what ships with every release.

19.1 Formats

FORMAT	FOR	NOTE
JSONL	general, model training	one corpus item per line, tags nested
CoNLL-U	linguistic tooling	token-level, the standard everyone in NLP already reads
TMX	localisation industry	Chosen on purpose — it's what translation-memory tools already consume, so there's no adoption friction for the commercial audience.
CSV	teachers, journalists, non-technical users	flattened and lossy, and labelled as such
HF Datasets	the ML ecosystem	versioned, with a dataset card

19.2 Export tiers by rights

full	everything, research use, non-commercial
clean	consented contributions only, commercially usable
regional-*	per-region slices
lexicon	lexicon entry / variant / form only, no source text
etymology	lexicon entry, origin, source form – small, self-contained, and probably the most citable slice available early

19.3 Every release ships with

Version number, checksum, corpus item counts by region and unit. The rule version used for canonical forms. Agreement statistics, reported honestly. A changelog against the previous version. A dataset card covering collection method, known biases, intended and unintended uses. A Zenodo DOI.

Build sequence

Five phases, and a rule about where this is allowed to stop.

WHERE PHASE 1 ACTUALLY STANDS, THIS REVISION

The schema, the task engine, consensus, and the corpus-items ingestion pipeline are built and exercised against a running database. Nothing user-facing is — no review queue UI, no scraper against a real source yet. The checklist below marks each line rather than restating that split in prose.

Phase 1 — Core pipeline · goal: a working annotation loop

- ☐ Monorepo scaffold: `control-plane/` (backend + core under `backend/core`, `worker`, `scrapers`), `data-plane/`, `intelligence-plane/nlp`, `frontend`, `shared`, `db` — **done**. `worker/` is a real package with `pg-boss` wired up and one live job; `scrapers/` is bootstrapped as a package but its `pipeline/` and `sources/` folders are still empty (§10.0)
- ☐ Prisma schema: `documents`, `corpus_items`, `tokens`, `links`, `tags`, `tasks`, `users`, `sources`, `audit_log` — **done**, and it now also covers every phase-2/3/5 table (`lexicon`, `translations`, `corrections`, `flags`, `snapshots`, `publications`, `outbox`, `stored objects`) — cheaper to declare an empty table now than migrate one in later
- ☐ `lexicon_origins` and `origin_enum` in the schema from the first migration, even though nothing writes to them until phase 2 (§7.6) — **done**
- ☐ Migrations — **done**; hand-written seed data from all four regions — **not started** (`db/seeds/` is still an empty stub)
- ☐ Auth: reviewer login, session cookies, guard chain — **partly done**. Email/password with `argon2id`, JWT access+refresh with rotation and reuse detection, CSRF double-submit, and the full `ThrottlerGuard` → `AuthGuard` → `RolesGuard` → `TrustGuard` chain are built and live-verified (§9.5). GitHub OAuth isn't wired yet.
- ☐ Task engine: `claim`, `complete`, `reject`, `rework`; spawn rules; idempotency keys from the start — **done**, and `review/region_tag/confirm` completions are live-verified end to end (§8.1). `translate_*`, `transliterate_*`, `standardise`, `adjudicate`, `link_lemma` completions still need the `lexicon/translate` modules.
- ☐ Adjudication routing and `target_regions` — from phase 1, not later. The routing rule is what protects the corpus, and retrofitting it once reviewers have habits is worse than useless. **The routing logic itself is built** (§8.5) and fires automatically below the agreement threshold; the phase 1 *interface* is the admin panel, which doesn't exist yet — the dedicated adjudication screen comes in phase 3.
- ☐ Review queue UI, fully keyboard driven — **not started**
- ☐ `CorpusItemText`, `RegionPicker`, `TaskCard`, `TagChip` — **not started** (§15.3)
- ☐ One scraper against one forum source, end to end — not YouTube (§12.1) — **not started**; the ingestion pipeline functions it'll run through (`clean`, `length filter`, `marker filter`, `near-duplicate`, `split into corpus items`, `compute match_key`) are already written in `control-plane/backend/core/src/ingestion/` and `text/`, just not driven by a real scraper yet
- ☐ Consensus computation + adjudication flagging — **done**, live-verified
- ☐ `pg-boss` with poll interval and concurrency capped from the first commit (§10.3) — **partly done**: `pg-boss` is wired in `worker/` against `DIRECT_URL`, one job live (`expire_claims`, 5 min); the rest of §9.3's job list and the poll/concurrency caps themselves are still to come
- ☐ `intelligence-plane/nlp` — FastAPI, health endpoint only, containerised, in compose — **done** (§11)
- ☐ `data-plane` — Actix, health endpoint only, containerised, in compose — **done** (§7.11)
- ☐ Both reachable from the backend, containerised and healthchecked — **done**; shown on an admin-overview widget — **not started**, since the admin overview itself doesn't exist yet (§17.1)
- ☐ Docker Compose running the whole thing locally — **done** (`docker-compose.local.yml` for dev infra, `docker-compose.prod.yml` for the full stack)

PHASE 2 DOES NOT END WITH A DATASET

The obvious phase 2 goal is "something citable exists" — a download page and a DOI. Right artifact for a researcher, wrong one for everybody else, and everybody else is who this needs. Nobody landing on this site downloads a JSONL. They type a word into the translator. If it answers, they might contribute. If the front page is a download link and a methodology essay, they leave.

Every hard problem in §5 — reviewer recruitment, regional balance, staying alive — is a problem of getting people to show up and keep showing up. A dataset can't do that. A tool that answers a question they actually had can. So phase 2 ships the translator, the compare view and the coverage page as a working site. The published snapshot is a byproduct, not the point.

Phase 2 — A working site · goal: something a Tunisian can actually use

Phase 2 proper hasn't started — the lines below are still the plan, except three pieces of infrastructure that got built early because phase 1's build sequence needed them anyway.

- ☐ Lexicon: lexicon_entries, lexicon_variants, lexicon_variant_regions, lexicon_forms — the translator runs on it. Schema done (§7.5); no module, no endpoints yet.
- ☐ Word origin UI on the lexicon entry page, plus the /origins browse-by-layer page
- ☐ Publications page, seeded with the prior work already cited in §1.2 and §13.1
- ☐ Lemma-linking workflow and review screen — core annotation work
- ☐ **Translator page:** word and sentence lookup, region-filtered, coverage-honest (§14)
- ☐ pg_trgm + match_key normalisation live (§13.5) — the translator has nothing to run on without it. match_key is already computed on every contributed corpus item (§8.1's corpus-items module); the lexicon side isn't built yet.
- ☐ Translator miss log feeding the gaps data
- ☐ Public browse and search across all the filter axes
- ☐ Entry permalink pages with revision history and the report/flag button (§16.4) — plus a bare list view for me to see and resolve flags, since a report button with nobody looking at what it collects isn't a moderation plan. The full unified lane merging flags in with corrections, origins and publications (§17.2) is a phase 3 nicety, not the phase 2 minimum.
- ☐ Compare view across four regions
- ☐ Coverage dashboard, public
- ☐ Methodology, style guide, licence, citation pages
- ☐ File storage interface wired: presigned URLs against R2, dev uses local disk (§9.5) — **built early** as an interface plus both adapters (control-plane/backend/core/src/storage/); not yet called from any route
- ☐ Export pipeline: JSONL + HF Datasets, checksummed and versioned, served via presigned URLs not routed through the backend
- ☐ First snapshot published with a dataset card and a Zenodo DOI
- ☐ Consent revocation + anonymisation routine, tested (§4.3)
- ☐ Backup encryption wired in before the first real backup runs, not after (§2.3) — **built early:** devops/backup/pg-backup.sh gpg-encrypts before anything leaves the box
- ☐ CI: master.yml (lint/typecheck/test/migration-drift/CodeQL) and verify-backup-restore.yml — **built early**, since CI has to exist before there's anything worth protecting with it; infra-watch.yml still not built
- ☐ **Decide hosting and object storage here, not before** (§2.1, §2.5) — size against measured usage, deploy, TLS, backups running and restore-tested

Phase 3 — Community · goal: contribution that doesn't need me

- ☐ Anonymous contribution flow with consent capture
- ☐ Confirm / vote page — the low-friction one that converts visitors
- ☐ Task inbox for contributors, gated by trust level
- ☐ Translation and transliteration task UIs
- ☐ Correction proposals + review flow, sharing the lane with origin and publication proposals
- ☐ Leaderboard by approved contributions and by region

- ☐ Gaps page driven by real coverage deficits and translator misses
- ☐ Trust-level computation job
- ☐ Admin dashboard: overview, users, sources, tasks, snapshots, privacy operations

Phase 4 — Service layer and machine assistance · only once the corpus earns it

- ☐ Standardisation rule engine + versioned regeneration, with rollback via prior rule_version (§17.2)
- ☐ Embeddings via hosted API + pgvector; fuzzy translation memory retrieval
- ☐ Public API, keys, rate limiting, usage dashboard
- ☐ API documentation
- ☐ TMX and CoNLL-U export formats
- ☐ **Dialect pre-filter** — batch inference on serverless GPU, off-box (§11.3)
- ☐ **Regional classifier** trained on the corpus itself — the research contribution
- ☐ Model registry and evaluation against frozen splits
- ☐ PostHog (free tier, §10.0) — I can't tell if the translator is used without it, and reusing a tool I already know beats re-evaluating analytics platforms from zero

Phase 5 — Voice and generation · furthest out, and genuinely optional

- ☐ Attestation audio: record a clip of a variant, play it back on the lexicon entry (§7.11)
- ☐ speakers and media tables populated for the first time
- ☐ Object storage re-picked on egress cost, not backup cost (§2.5)
- ☐ Voice consent enforcement and the deletion path (§4.3)
- ☐ Grounded generation for unattested translator input, labelled clearly as generated (§14.6)

Not building

audio beyond phase 5 attestation clips — no ASR, no TTS, no speech models

morphological annotation

any model before phase 4

on-server inference

mobile apps

paid tiers

subregions

All accounted for in the schema — nullable columns, stub tables, the `is_machine` and `model_versions` structures already sitting there — so adding any of them later is a migration, not a rewrite.

THE RULE I'M HOLDING MYSELF TO

Phases 1 and 2 are scoped to produce a complete, publishable, citable thing on their own. **If this stops after phase 2, it has still produced something that didn't exist before.** Everything after that is upside, not a requirement for the work having been worth doing.

OpenDerja_TN · Firas Lamouchi · planning started 14 April 2026, build started 28 May 2026, updated 25 August 2026 after moving the repository to a remote GitHub host

Companion documents: Architecture & Engineering Blueprint · VOICE.md